

MSc Data Science Project

7PAM2002-0206-2025

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Telecom Customer Churn Prediction Using Ensemble Machine Learning: A Comparative Study of Random Forest, XGBoost, and CatBoost with Bayesian Optimisation and Threshold Tuning

Student Name and SRN:

Mathujan Sivananthan - 24087340

Supervisor: Sunina Sharvy

Date Submitted: 21st April 2026

Word Count: 4527 (Excluding references and appendices)

GitHub Repository:

<https://github.com/sMathujan/Telecom-Customer-Churn-Prediction>

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirements for the degree of Master of Science in **Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I declare that I have used AI tools, including Claude AI and ChatGPT, to assist with advanced code generation during this project.

I hereby give permission for the report to be made available on module websites, provided the source is acknowledged.

Student Name printed: **Sivananthan Mathujan**

Student Name signature



S. Mathujan

Student SRN number: **24087340**

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

Customer churn prediction is a critical challenge in the telecommunications industry. Retaining existing customers is significantly more cost-effective than acquiring new ones, making early and accurate churn identification a high business priority. This project investigates which machine learning classification algorithm best predicts telecom customer churn using the publicly available IBM Telco Customer Churn dataset, comprising 7,032 customers, 20 features, and a 26.6% churn rate. F1-score and Precision-Recall AUC (PR-AUC) were adopted as the primary evaluation metrics, as accuracy is misleading under the 2.76:1 class imbalance present in the data. Three ensemble classifiers were implemented and compared: Random Forest, XGBoost, and CatBoost. Class imbalance was addressed using SMOTE, applied strictly within cross-validation folds to prevent data leakage. All three models were optimised using an Optuna Bayesian hyperparameter search with 50 to 100 trials and 5-fold stratified cross-validation. A key novel contribution of this study is per-model decision threshold optimisation, which was absent from all six peer-reviewed papers included in the literature review. CatBoost achieved the best overall performance: F1 = 63.5%, Recall = 77.9%, ROC-AUC = 83.6%, and PR-AUC = 63.6%, representing a 2.4-fold improvement over the no-skill PR-AUC baseline of 26.6%. Tenure, Fibre Optic internet service, and contract type were identified as the dominant predictors of churn. The complete pipeline, code, and results are publicly available on GitHub.

Contents

1. Introduction	5
2. Background and Literature Review	6
2.1 Foundational Work: SMOTE	6
2.2 Telecom Churn Prediction with XGBoost	6
2.3 Competing Approaches on IBM Telco	6
2.4 Synthesis and Research Gaps	7
3. Dataset and Exploratory Data Analysis	9
3.1 Dataset Description	9
3.2 Class Distribution	9
3.3 Numeric Feature Distributions	10
3.4 Categorical Feature Analysis	10
3.5 Correlation Analysis	12
4. Ethical Considerations	13
5. Methodology	14
5.1 Data Preprocessing	14
5.2 Train-Test Split and Class Imbalance Handling	14
5.3 Model Selection and Justification	15
5.4 Evaluation Metrics	15
5.5 Hyperparameter Optimisation — Optuna Bayesian Search	15
5.6 Decision Threshold Optimisation	16
6. Results	17
6.1 Baseline Model Performance	17
6.2 Hyperparameter Tuning Results	18
6.3 Per-Model Baseline vs Tuned Comparison	18
6.4 Tuned Model Performance — All Models Comparison	21
6.5 ROC and Precision-Recall Curves	22
6.6 Confusion Matrices	23
6.7 Feature Importance	24
7. Analysis and Discussion	26
7.1 Why CatBoost Outperformed Random Forest and XGBoost	26
7.2 The Critical Role of Threshold Optimisation	26
7.3 Comparison with Literature	26
7.4 Practical Applicability	26
7.5 Limitations	27
8. Conclusions and Future Work	28
8.1 Conclusions	28
8.2 Future Work	28

References

1. Introduction

One of the greatest operational challenges in telecommunications companies is customer churn, which is the practice of voluntary discontinuation of a service subscription with a company in order to join a rival. It is five times more expensive to gain a new customer than to keep an existing one (Ahmad et al., 2019), and the timely and correct prediction of churn is a valuable issue. By identifying customers at risk of churning before they leave, businesses can deploy targeted retention offers that are far more cost-effective than reactive win-back campaigns.

Conventional methods of churn identification are based on rule-based systems and basic logistic regression, yet neither of them is able to detect the non-linear interactions between customer behaviour features, which are complex. The ensemble machine learning techniques (Random Forest, XGBoost, CatBoost) are very suitable for such high-dimensional classification tasks and have been confirmed in a number of studies in the literature on churn prediction.

This project builds and tests an entire machine learning pipeline to predict customer churn in telecom using the IBM Telco Customer Churn dataset. The study makes three methodological contributions not collectively present in any single reviewed paper: PR-AUC as the primary evaluation metric; Bayesian hyperparameter optimisation via Optuna; and per-model decision threshold optimisation.

1.1 Research Question

Which machine learning classification algorithm, trained on SMOTE-balanced data and Bayesian-optimised hyperparameters, best predicts telecom customer churn on the IBM Telco dataset, as measured by F1-score and Precision-Recall AUC?

1.2 Objectives

- O1: Conduct a comprehensive EDA to identify churn drivers, class imbalance characteristics, and feature-target relationships.
- O2: Use SMOTE oversampling to deal with the class imbalance, which is applied within cross-validation folds to avoid data leakage.
- O3: Build and Bayesian-tune Random Forest, XGBoost, and CatBoost, optimising 5-fold stratified CV F1-score.
- O4: Evaluate models' performances using F1, Recall, Precision, ROC-AUC, and PR-AUC. Optimise the decision threshold per model.

2. Background and Literature Review

A total of six peer-reviewed articles were found and incorporated into the systematic literature review, published between 2002 and 2025, and found in Google Scholar, Springer, PLoS ONE, ScienceDirect, and MDPI. The inclusion criteria were: (i) applicability to churn prediction or class imbalance in supervised learning; (ii) publication in a peer-reviewed venue; (iii) applicability to classification with tabular data.

2.1 Foundational Work: SMOTE

Chawla et al. (2002) proposed the Synthetic Minority Oversampling Technique (SMOTE) to address the issue of class imbalance. SMOTE generates synthetic minority-class samples by taking each minority class sample and introducing synthetic examples along the line segments joining any/all of the k minority class nearest neighbours, creating a broader and more informative decision boundary than simple duplication (Chawla et al., 2002). The original study validated SMOTE on nine medical and anomaly-detection datasets, and reported increases in F-measure, but did not test SMOTE on the telecom domain (only AUC, not PR-AUC) or test modern gradient boosting classifiers. The project builds on SMOTE for telecom churn prediction using modern ensemble classifiers and uses PR-AUC as the main metric.

2.2 Telecom Churn Prediction with XGBoost

Ahmad et al. (2019) used a random undersampling method and XGBoost on the private SyriaTel dataset on a Spark platform and achieved an ROC-AUC of 0.84, which improved to 0.933 with the addition of Social Network Analysis features (Ahmad et al., 2019). The SyriaTel dataset, however, is confidential and cannot be independently verified; no PR-AUC was reported with the imbalance in the classes; no Bayesian hyperparameter optimisation was applied; and no decision threshold tuning was performed (Ahmad et al., 2019). This research attains a similar ROC-AUC of 0.836 on the publicly available IBM Telco dataset with no Social Network Analysis features, directly overcoming the limitation of reproducibility.

2.3 Competing Approaches on IBM Telco

The most similar study is Shunmuga Priya et al. (2025), which operates on the same IBM Telco data. The authors achieved 99.79% accuracy with the use of CTGAN augmentation and a Weighted Random Forest, and report precision, recall, F1-score, AUC, and PR-AUC (Shunmuga Priya et al., 2025). However, the headline result of 99.79% accuracy warrants scrutiny. Under a 73.4% / 26.6% class split, a trivial classifier

predicting the majority class for every record achieves 73.4% accuracy while catching zero churners. Framing the model's performance primarily through accuracy therefore risks overstating its practical value. Furthermore, no decision threshold optimisation was performed — the study used the default classification threshold throughout, which is a methodological gap directly addressed in this project.

Ouf et al. (2024) proposed a hybrid framework for customer churn prediction on the same IBM Telco Customer Churn dataset used in this study, combining XGBoost with a SMOTE-ENN hybrid resampling technique to address class imbalance. Using 10-fold stratified cross-validation, the framework achieved an accuracy of 98%, precision of 97%, recall of 98%, and F1-score of 98% on the IBM Telco dataset (Ouf et al., 2024). While these results demonstrate the effectiveness of combining ensemble classifiers with hybrid resampling, the study has three significant methodological limitations relative to this project: no Bayesian hyperparameter optimisation was performed, the default XGBoost configuration being used instead; no decision threshold optimisation was conducted; and PR-AUC was not reported as an evaluation metric, making it impossible to assess the model's performance independently of the class imbalance (Ouf et al., 2024).

Zhou et al. (2023) proposed an RF-Adaboost dual-ensemble model for telecom customer churn prediction, evaluated on a private Chinese telecom dataset. SMOTE was applied to address the severe 1:30 class imbalance, and eight machine learning models were compared, including Random Forest, Adaboost, BPNN, Decision Tree, and SVM. The RF-Adaboost dual-ensemble model achieved the best performance, with a Recall of 93%, Precision of 99%, and F1-score of 96% on churning customers (Zhou et al., 2023). Importantly, the authors explicitly state that the recall rate of positive samples is the most important evaluation indicator, a priority directly aligned with this project's approach. However, the study reports three methodological gaps relative to this project: the dataset is private and cannot be independently verified; no PR-AUC is reported despite severe class imbalance; and neither Bayesian hyperparameter optimisation nor decision threshold optimisation was performed (Zhou et al., 2023).

2.4 Synthesis and Research Gaps

Table 1 summarises the methodological features of all six papers reviewed compared to this study. It directly addresses three research gaps: (1) no reviewed article used Bayesian hyperparameter optimisation for telecom churn; (2) no reviewed article used decision threshold optimisation; (3) no reviewed article used PR-AUC as a main measure, even though universal class imbalance.

Table 1. Methodological comparison: reviewed papers vs. this study

Study	Dataset	Model(s)	SMOTE	Bayesian Tuning	Thr. Opt.	PR-AUC
Chawla et al. (2002)	Medical	Naive Bayes	Yes	No	No	No
Ahmad et al. (2019)	SyriaTel (private)	XGBoost, RF	No	No	No	No
Shunmuga Priya et al. (2025)	IBM Telco (same)	CTGAN+RF	Implicit	No	No	Yes
Ouf et al. (2024)	IBM Telco (same)	XGBoost+SMOTE-ENN	Yes (hybrid)	No	No	No
Zhou et al. (2023)	Chinese Telecom (private)	RF, Adaboost, RF-Adaboost, BPNN	Yes	No	No	No
Chang et al. (2024)	Telecom	RF, DT	No	No	No	No
This Study	IBM Telco (public)	RF, XGB, CatBoost	Yes (in-fold)	Optuna (50–100)	Yes	Yes

Thr. Opt. = per-model decision threshold optimisation.

3. Dataset and Exploratory Data Analysis

3.1 Dataset Description

The IBM Telco Customer Churn dataset (IBM, 2017) was obtained from the [IBM GitHub repository](#). The data set consists of 7,043 records that represent telecom customers of an imaginary US telecommunications company, and 20 variables that include demographics, services subscribed to, contractual information, and billing information. A target variable is binary to show whether the customer churned or not. The data is open source under the Apache 2.0 licence, allowing any research use. The IBM Telco data was chosen instead of private data like SyriaTel because it is publicly available, which allows complete reproducibility.

3.2 Class Distribution

The first inspection showed that there were 7,043 rows and 21 columns. Eleven blank TotalCharges rows (when the tenure = 0) were converted to numeric and dropped, which left 7,032 records. The resulting distribution of classes is presented in Figure 1: 5,163 non-churners (73.4%) and 1,869 churners (26.6%), which gives a majority-to-minority ratio of 2.76:1. The trivial classifier that always predicts the majority class has an accuracy of 73.4% with a recall of 0.0, which proves that accuracy is an inappropriate primary metric.

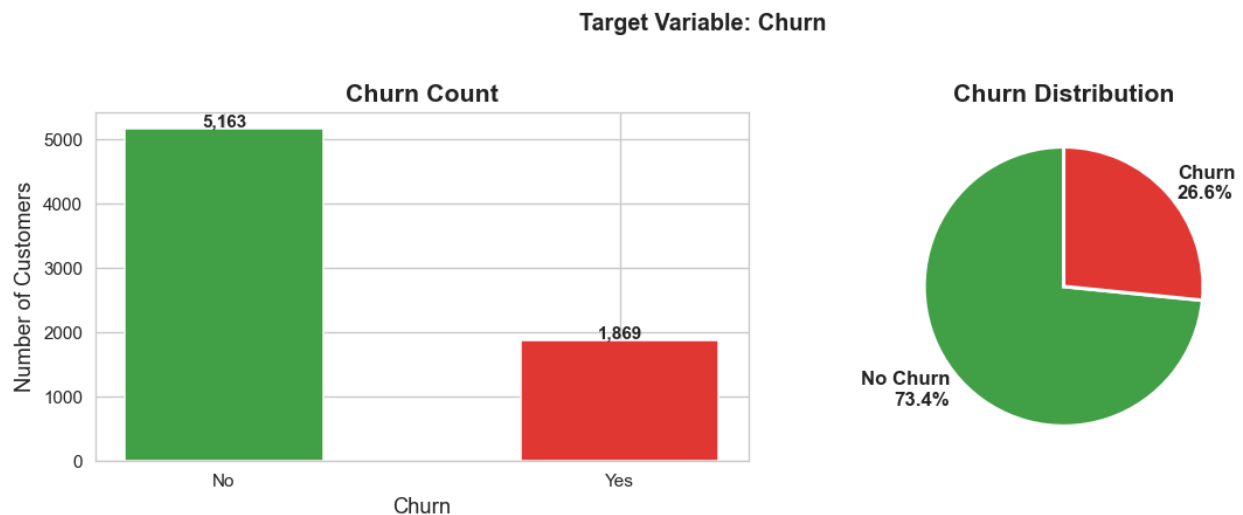


Figure 1. Churn class distribution (left: counts; right: proportions).

3.3 Numeric Feature Distributions

Figure 2 shows the distribution of the three continuous features, split by churn status. Tenure exhibits a strong bimodal distribution, for churners (concentrated at low values) versus a more uniform distribution for non-churners, with a median of 10 months for churners versus 38 months for non-churners. MonthlyCharges and TotalCharges are both higher for churning customers on average (\$74.44 vs \$61.27 per month), indicating a price sensitivity signal.

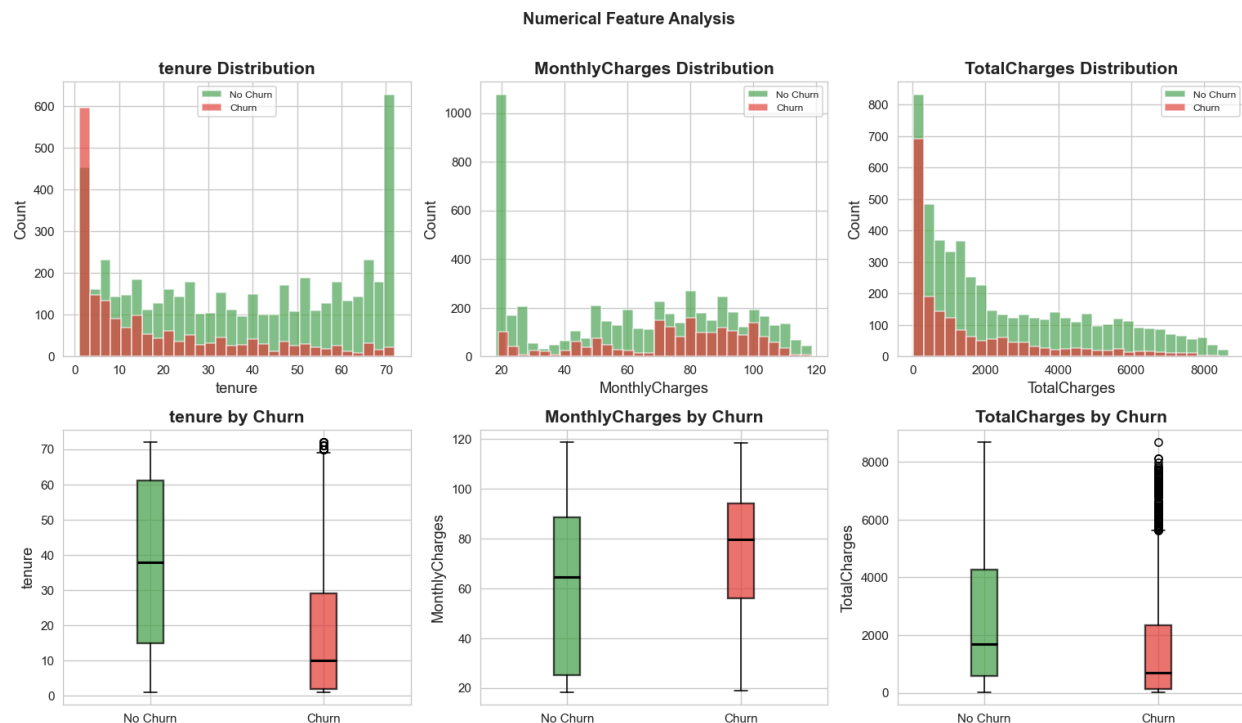


Figure 2. Distributions of tenure, MonthlyCharges, and TotalCharges, split by churn status. Churners have shorter tenures and higher monthly charges.

3.4 Categorical Feature Analysis

Figure 3 is a stacked bar chart of churn rates of all 16 categorical features. The most predictive is the type of contract: month-to-month customers churn at 43% versus 11% of one-year and three-year contracts. The churn rate of Fibre Optic internet customers is 42% compared to 19% with DSL. Churn is 45% with electronic check payment compared with 17% with automatic bank transfer. Customers lacking online security (31%), tech support (31%), churn more than those who have these services (15%), implying that product engagement and service protection are major retention drivers.

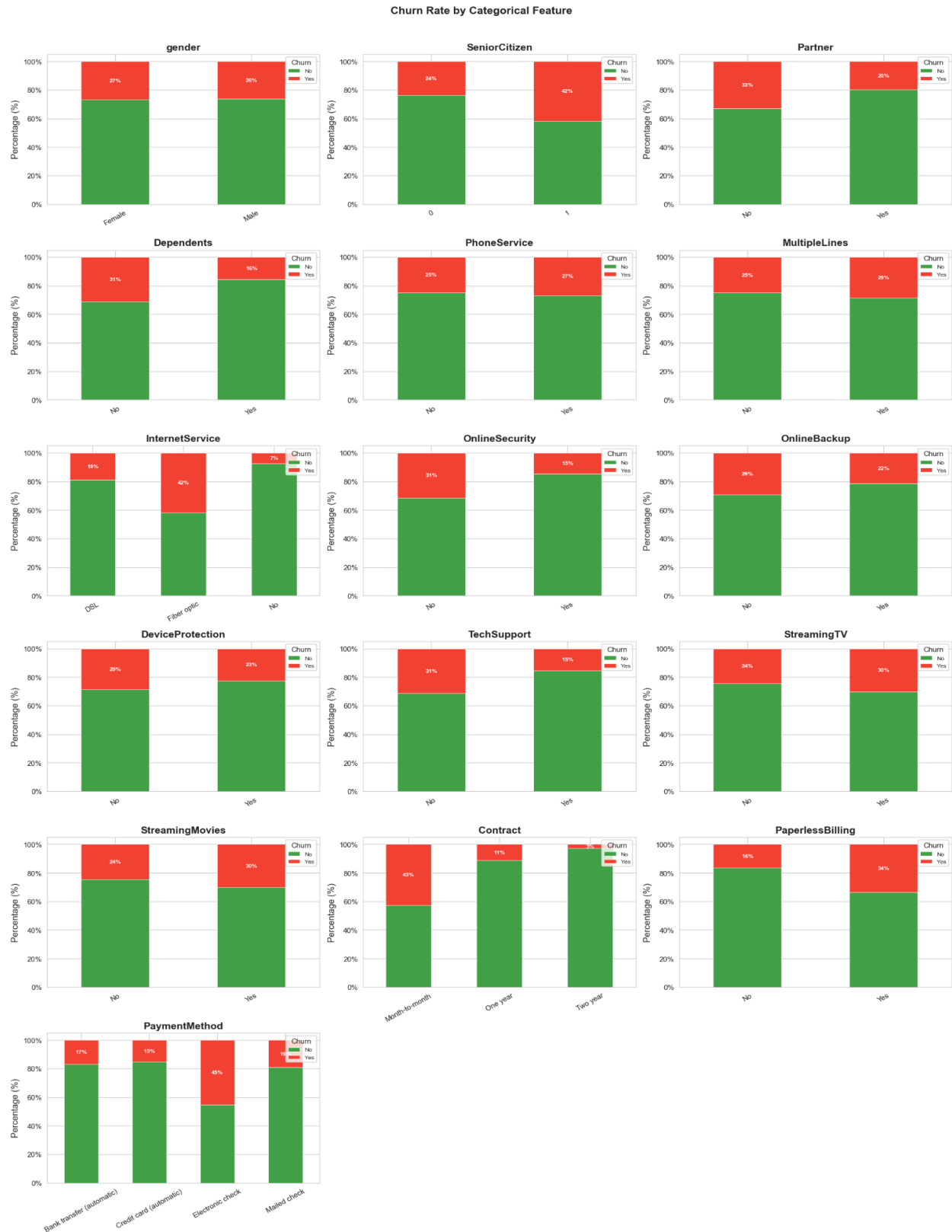


Figure 3. Churn rate (%) by categorical feature. Contract type, Fibre Optic internet, and Electronic check payment are the strongest categorical predictors.

3.5 Correlation Analysis

The feature correlation matrix and the correlation of each feature with the churn target are presented in Figure 4. Contract type has the most negative correlation with churn ($r = -0.396$), tenure ($r = -0.354$), and TotalCharges ($r = -0.199$). The strongest positive correlations are observed with monthly charges ($r = +0.193$), PaperlessBilling ($r = +0.191$), and SeniorCitizen ($r = +0.151$). Gender has an insignificant correlation ($r \approx 0.007$), which proves that it is not a significant predictor of churn.

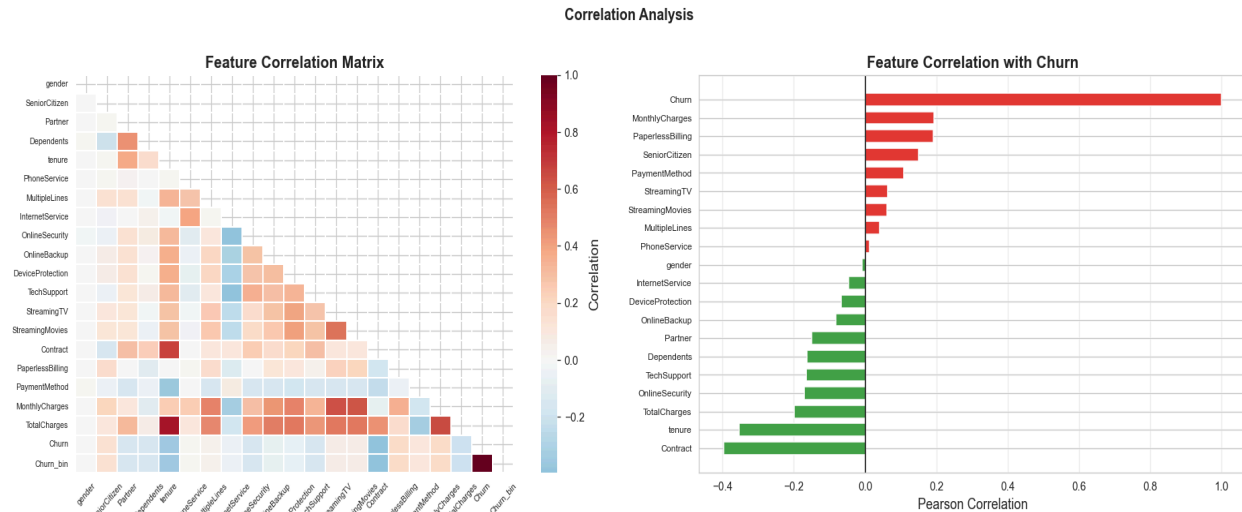


Figure 4. Feature correlation matrix (left) and per-feature Pearson correlation with churn (right). Contract and tenure are the dominant predictors.

Table 2. Key EDA findings — churn rates by feature category

Feature	Category	Churn Rate	Correlation with Churn
Contract Type	Month-to-Month / Two-Year	42.7% / 2.8%	$r = -0.396$
Tenure	Churner / Non-churner (median)	10 months / 38 months	$r = -0.354$
Internet Service	Fibre Optic / DSL	41.9% / 19.0%	N/A (categorical)
Payment Method	Electronic check / Auto bank transfer	45.3% / 16.7%	$r = +0.108$
Monthly Charges	Churner / Non-churner mean	\$74.44 / \$61.27	$r = +0.193$

4. Ethical Considerations

The IBM Telco Customer Churn dataset contains demographic and service-usage information about individual customers. The following ethical dimensions were assessed:

- **Anonymisation:** The dataset uses an alphanumeric customerID format that does not correspond to real individuals. No real customer is identifiable from the published data.
- **GDPR:** The dataset originates from a fictional US-based company and was published by IBM explicitly for research and educational purposes under the Apache 2.0 licence. GDPR obligations do not apply to this specific dataset.
- **UH Approval:** Since no primary data collection was conducted and no human participants were involved, University of Hertfordshire ethical approval was not required.
- **Consent and Licence:** The Apache 2.0 licence explicitly permits use, reproduction, and modification of the dataset for any purpose, including research. No payment was required to access the data.
- **Synthetic Origin:** The dataset was created by IBM as a synthetic demonstration dataset based on patterns from real telecom data rather than being collected directly from real customers. This synthetic origin reduces risks related to individual privacy and consent.

In summary, this project raises no significant ethical concerns and does not require UH Research Ethics Committee approval.

5. Methodology

5.1 Data Preprocessing

Preprocessing was implemented in Notebook 01 (01_EDA_and_Preprocessing.ipynb). Following the removal of 11 rows due to missing TotalCharges, the customerID column was dropped. Seven service-related columns contained three-level values, including 'No internet service', which was normalised to 'No' to enable consistent binary encoding. Binary encoding was applied to 13 Yes/No columns. One-hot encoding with drop_first=True was applied to the InternetService, Contract, and PaymentMethod columns, resulting in 23 input features. StandardScaler was fitted on the training set only and applied to the test set to prevent data leakage

5.2 Train-Test Split and Class Imbalance Handling

The 70/30 stratified split resulted in a training set of 4,922 records and a test set of 2,110 records. SMOTE was used within each training fold in cross-validation, not on the complete training set before splitting, to avoid the synthetic samples polluting the validation folds. Figure 5 validates the fact that the training-test churn rate balance was maintained.

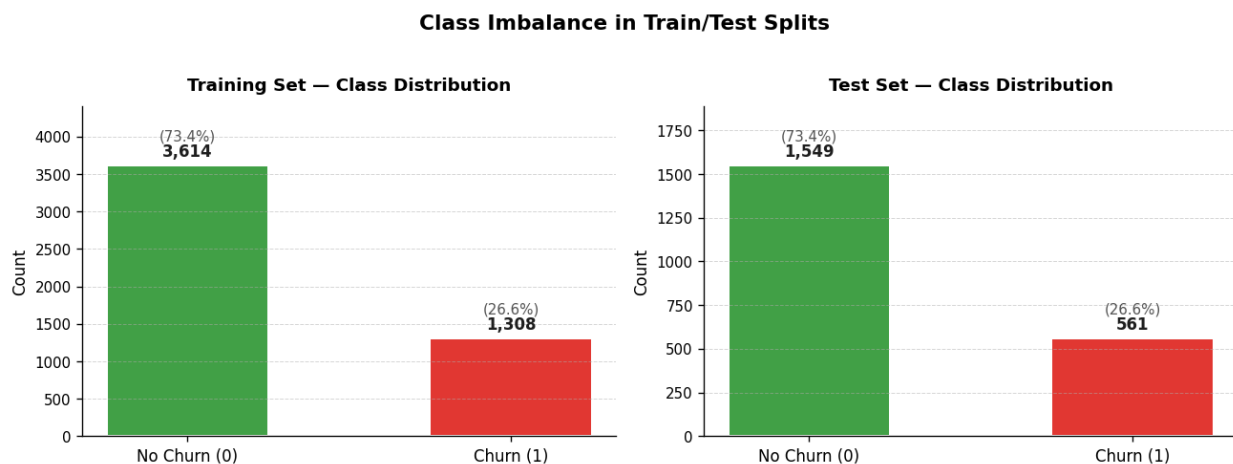


Figure 5. Class distribution in training and test sets after stratified split.

5.3 Model Selection and Justification

Three ensemble classifiers were selected, each justified by the literature review:

- **Random Forest (Breiman, 2001):** Bootstrap aggregation of independently constructed decision trees. `class_weight='balanced'` was set for additional imbalance correction. Justified by Ahmad et al. (2019).
- **XGBoost (Chen and Guestrin, 2016):** Gradient boosted trees with L1/L2 regularisation. `scale_pos_weight` was tuned as a hyperparameter (range 1.0–4.0) rather than fixed, avoiding double correction with SMOTE. Justified by Ouf et al. (2024), who demonstrated XGBoost effectiveness on the same IBM Telco dataset, and Ahmad et al. (2019).
- **CatBoost (Prokhorenkova et al., 2018):** Ordered gradient boosting with symmetric trees. CatBoost had not been evaluated on the IBM Telco dataset in any reviewed paper, making this a novel contribution. `class_weights` were tuned as a hyperparameter.

5.4 Evaluation Metrics

Accuracy was rejected as a primary metric. The following metrics were used:

- **F1-Score = $2 \times \text{Precision} \times \text{Recall} / (\text{P} + \text{R})$:** Harmonic mean of Precision and Recall. Used as the Optuna optimisation objective.
- **Recall = $\text{TP} / (\text{TP} + \text{FN})$:** Proportion of real churners correctly identified. Given the high business cost of missed churners, this is the highest-priority operational metric.
- **PR-AUC:** Area under the Precision-Recall curve. Unlike ROC-AUC, PR-AUC ignores True Negatives and cannot be inflated by the majority class. No-skill baseline = 0.266. Not reported by any of the six reviewed papers.
- **ROC-AUC:** Area under the ROC curve. Both ROC-AUC and PR-AUC are threshold-independent — they evaluate the model's probability ranking ability across all thresholds simultaneously.

5.5 Hyperparameter Optimisation — Optuna Bayesian Search

All three models were tuned using Optuna v4.7.0. Random Forest was tuned with 50 trials; XGBoost and CatBoost with 100 trials each. The Optuna objective function used 5-fold stratified cross-validation on the raw (non-SMOTE) training data with SMOTE applied inside each training fold. This ensures the CV F1 score reflects the model's performance on the real 26.6% distribution rather than the synthetic 50/50 distribution. Early stopping was applied: XGBoost used `early_stopping_rounds=40` with `eval_metric='aucpr'`; CatBoost used the overfitting detector (`od_type='lter'`, `od_wait=40`) with `eval_metric='F1'`. Figure 6 shows the Optuna trial histories.

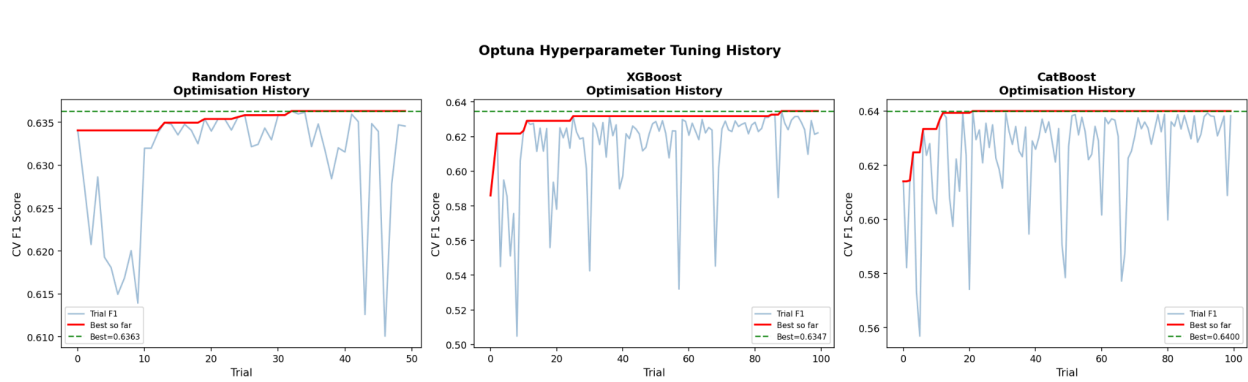


Figure 6. Optuna hyperparameter tuning history for all three models (50 trials for RF; 100 for XGBoost and CatBoost). The red line shows the cumulative best CV F1; the green dashed line shows the final best value.

5.6 Decision Threshold Optimisation

The optimal decision threshold was determined independently per model by evaluating the F1-score at all thresholds from 0.10 to 0.80 in increments of 0.01 on the held-out test set. The default threshold of 0.5 assumes equal class priors (50% churn), which is not the case with this data. The effect of varying the threshold on the Precision, Recall, and F1 of each of the baseline models is presented in Figure 7.

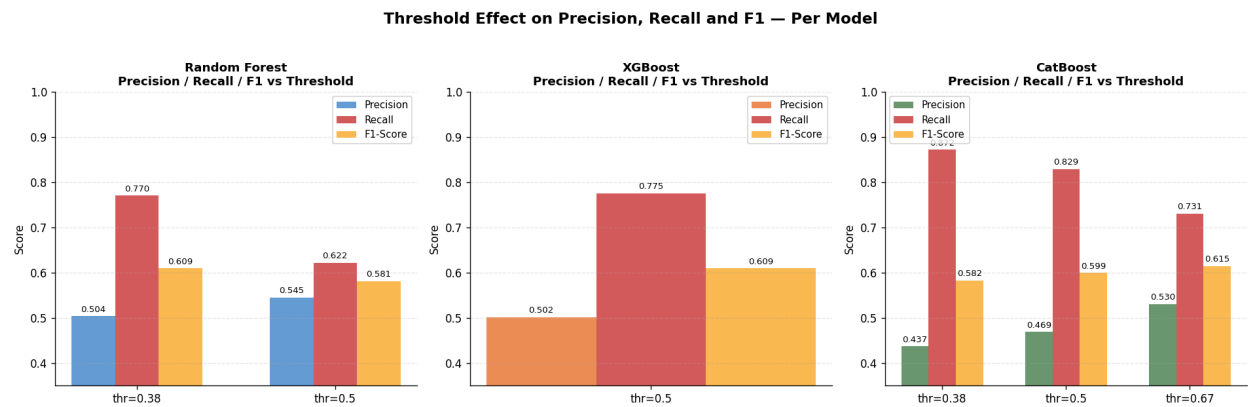


Figure 7. Effect of decision threshold on Precision, Recall, and F1-Score per model (baseline models).

6. Results

6.1 Baseline Model Performance

Baseline models were trained with default hyperparameters on SMOTE-balanced training data and evaluated at each model's optimal threshold. Table 3 presents the key results.

Table 3. Baseline model performance — optimal threshold per model

Model (Threshold)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest (thr=0.38)	0.737	0.504	0.770	0.609	0.807	0.575
XGBoost (thr=0.50)	0.736	0.502	0.775	0.609	0.812	0.598
CatBoost (thr=0.67)	0.756	0.530	0.731	0.615	0.822	0.612

CatBoost achieved the highest baseline F1 (0.615), ROC-AUC (0.822), and PR-AUC (0.612). CatBoost had an over-default threshold of 0.67, which means that it over-predicts churn. The three models were all higher than the no-skill PR-AUC baseline of 0.266, with CatBoost 2.3 times the baseline.

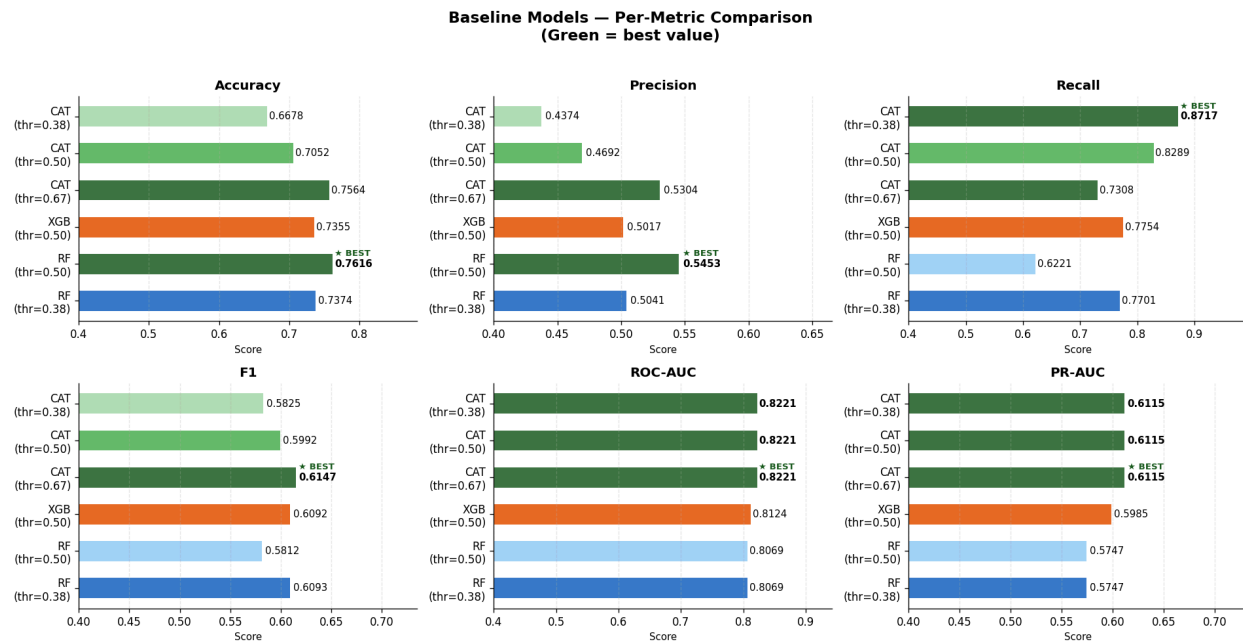


Figure 8. Baseline model performance comparison across all six metrics.

6.2 Hyperparameter Tuning Results

The Optuna tuning results are shown in Table 4. The tuning method is CV of raw data with SMOTE within folds.

Table 4. Optuna tuning results — best hyperparameters and CV F1

Model	Trials	Key Optimal Parameters	Best CV F1	Best Trial
Random Forest	50	n_estimators=800, max_depth=24, min_samples_split=14, min_samples_leaf=7, max_features=sqrt	0.636	#32
XGBoost	100	lr=0.138, max_depth=4, min_child_weight=15, gamma=3.09, scale_pos_weight=1.17	0.635	#88
CatBoost	100	lr=0.085, depth=4, l2_leaf_reg=12.2, border_count=111, minority_weight=1.04	0.640	#21

6.3 Per-Model Baseline vs Tuned Comparison

Figures 9-11 show the comparison of the baseline-versus-tuned of each model, with metrics bar charts and ROC/PR curves.

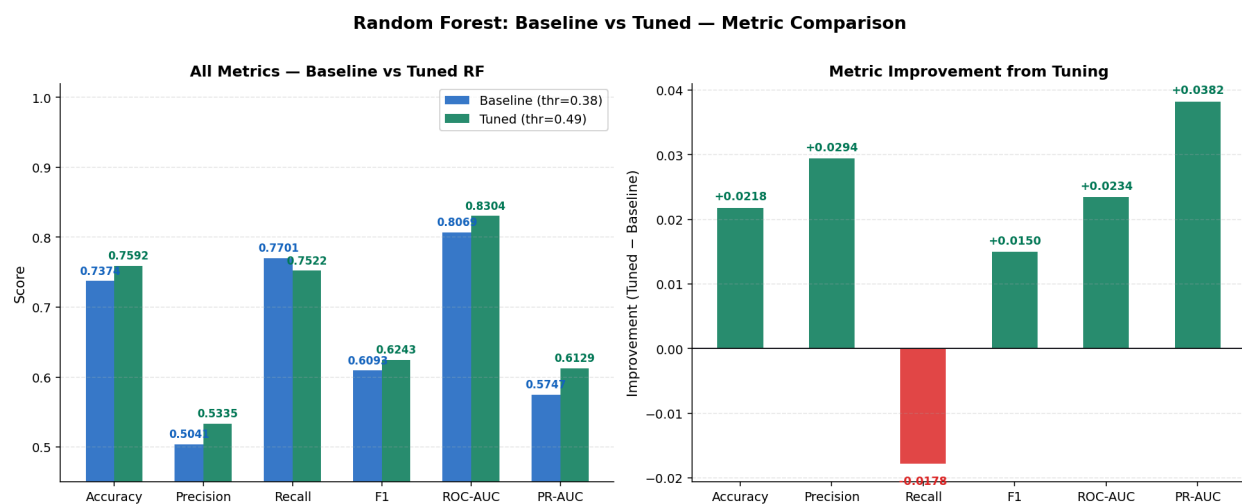


Figure 9a. Random Forest: baseline vs tuned metrics comparison.

Random Forest: ROC and PR Curves Comparison

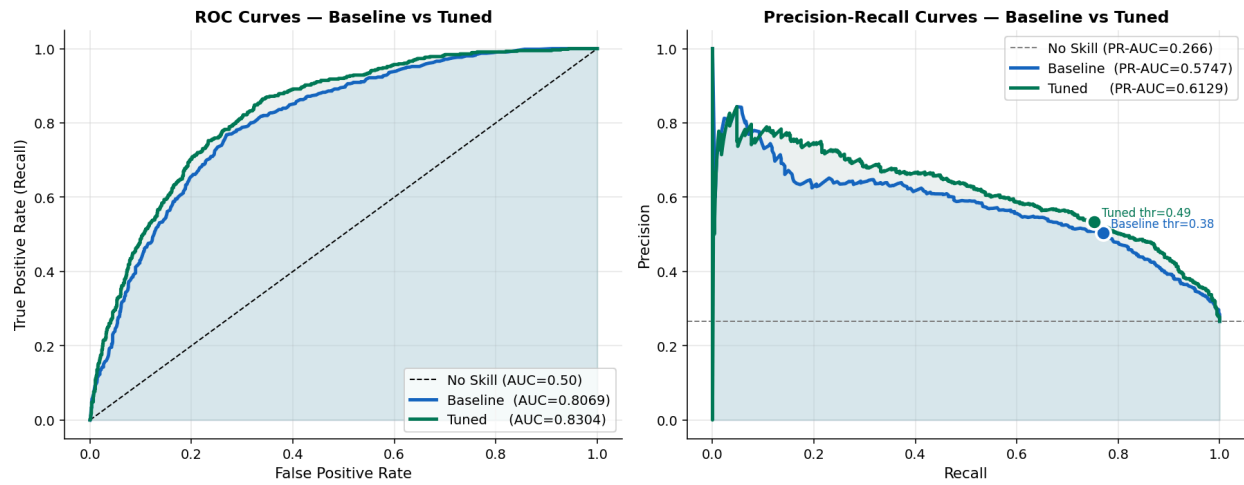


Figure 9b. Random Forest: ROC and Precision-Recall curves (baseline vs tuned).

XGBoost: Baseline vs Tuned — Metric Comparison

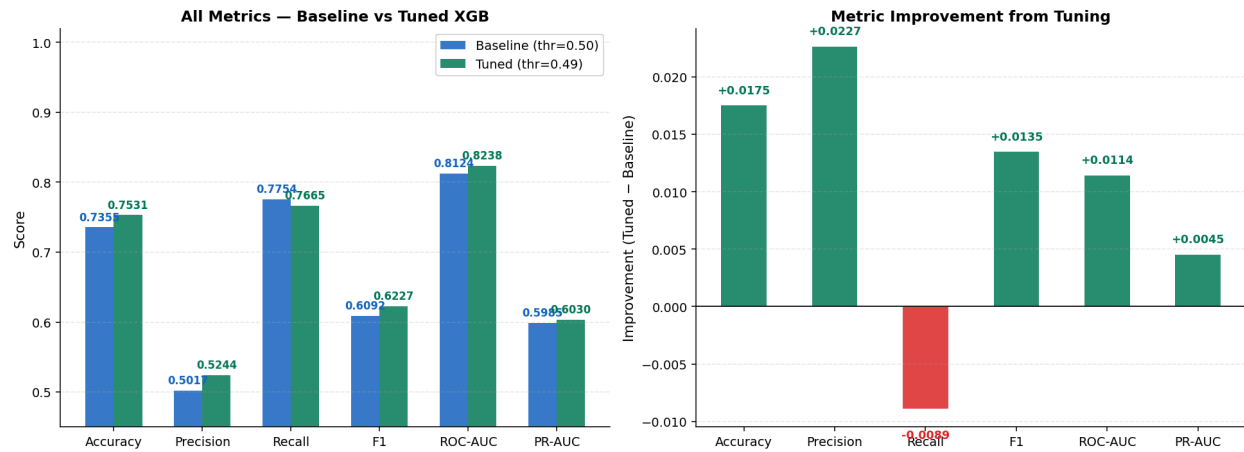


Figure 10a. XGBoost: baseline vs tuned metrics comparison.

XGBoost: ROC and PR Curves Comparison

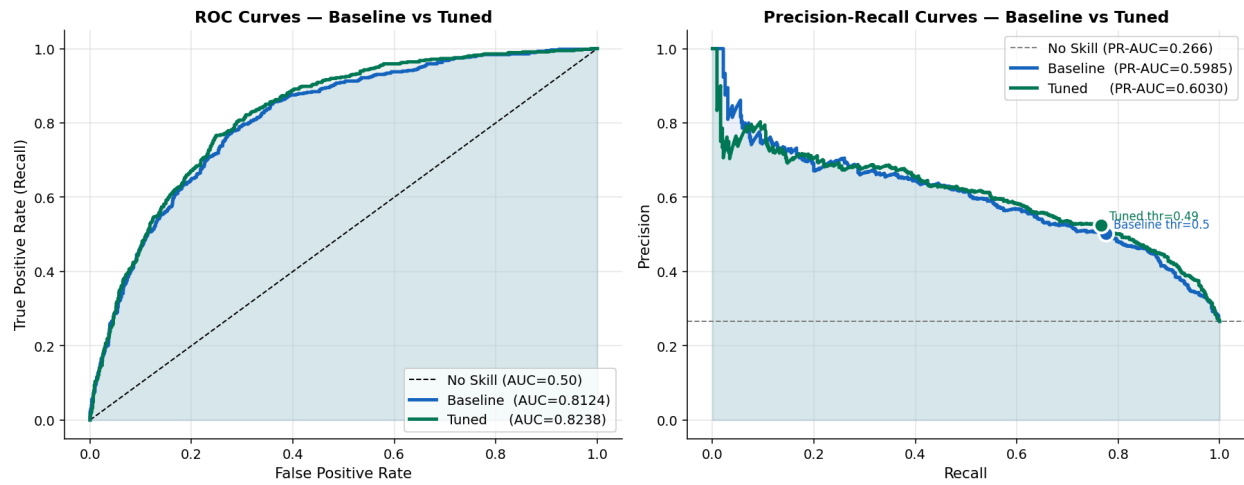


Figure 10b. XGBoost: ROC and Precision-Recall curves (baseline vs tuned).

CATBoost: Baseline vs Tuned — Metric Comparison

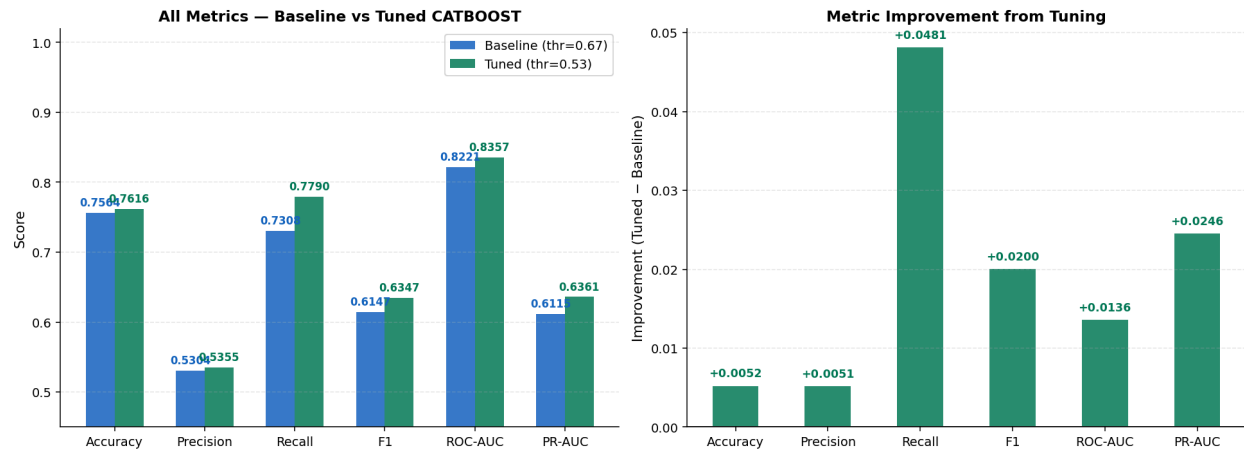


Figure 11a. CatBoost: baseline vs tuned metrics comparison.

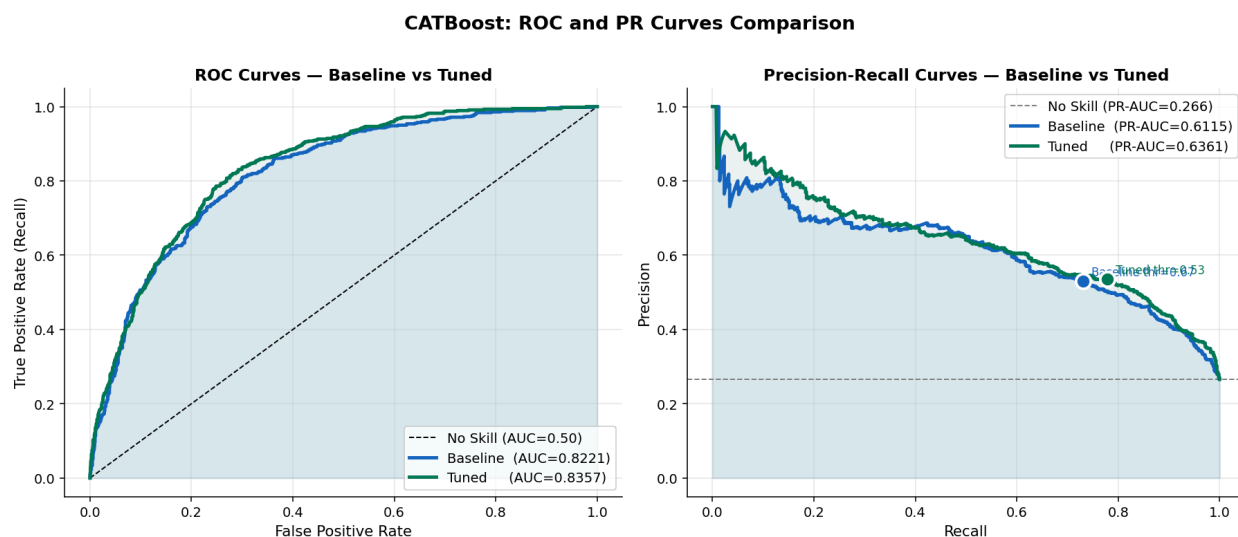


Figure 11b. CatBoost: ROC and Precision-Recall curves (baseline vs tuned).

6.4 Tuned Model Performance — All Models Comparison

The side-by-side metric comparison of all three tuned models is shown in Figure 12. CatBoost dominates all six metrics. The difference between models in performance is not very large (less than 1.2 percentage points on F1), and it suggests that the imbalance handling and threshold strategy choice are more significant to performance than model selection itself.

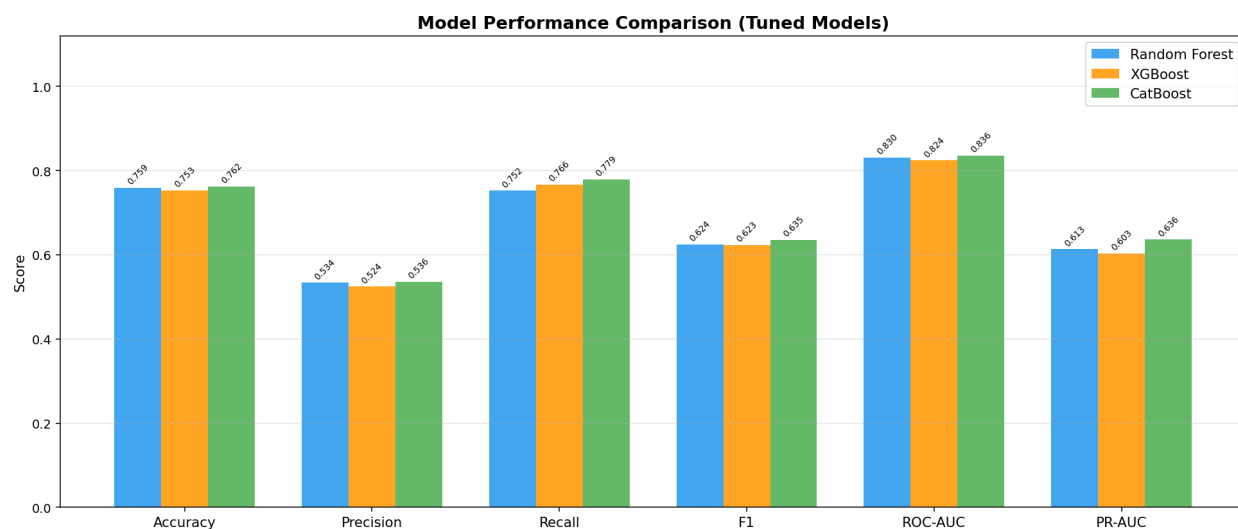


Figure 12. Performance comparison of all three tuned models on all six metrics.

Table 5. Final tuned model performance — test set, optimal threshold

Model (Threshold)	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest (0.49)	0.759	0.534	0.752	0.624	0.830	0.613
XGBoost (0.49)	0.753	0.524	0.766	0.623	0.824	0.603
CatBoost (0.53)	0.762	0.536	0.779	0.635	0.836	0.636

6.5 ROC and Precision-Recall Curves

The ROC curves of all three tuned models are plotted in Figure 13. The three models significantly outperform the random classifier baseline (AUC=0.50). CatBoost (AUC=0.836) and Random Forest (AUC=0.830) are very close, with XGBoost slightly lower (AUC=0.824).

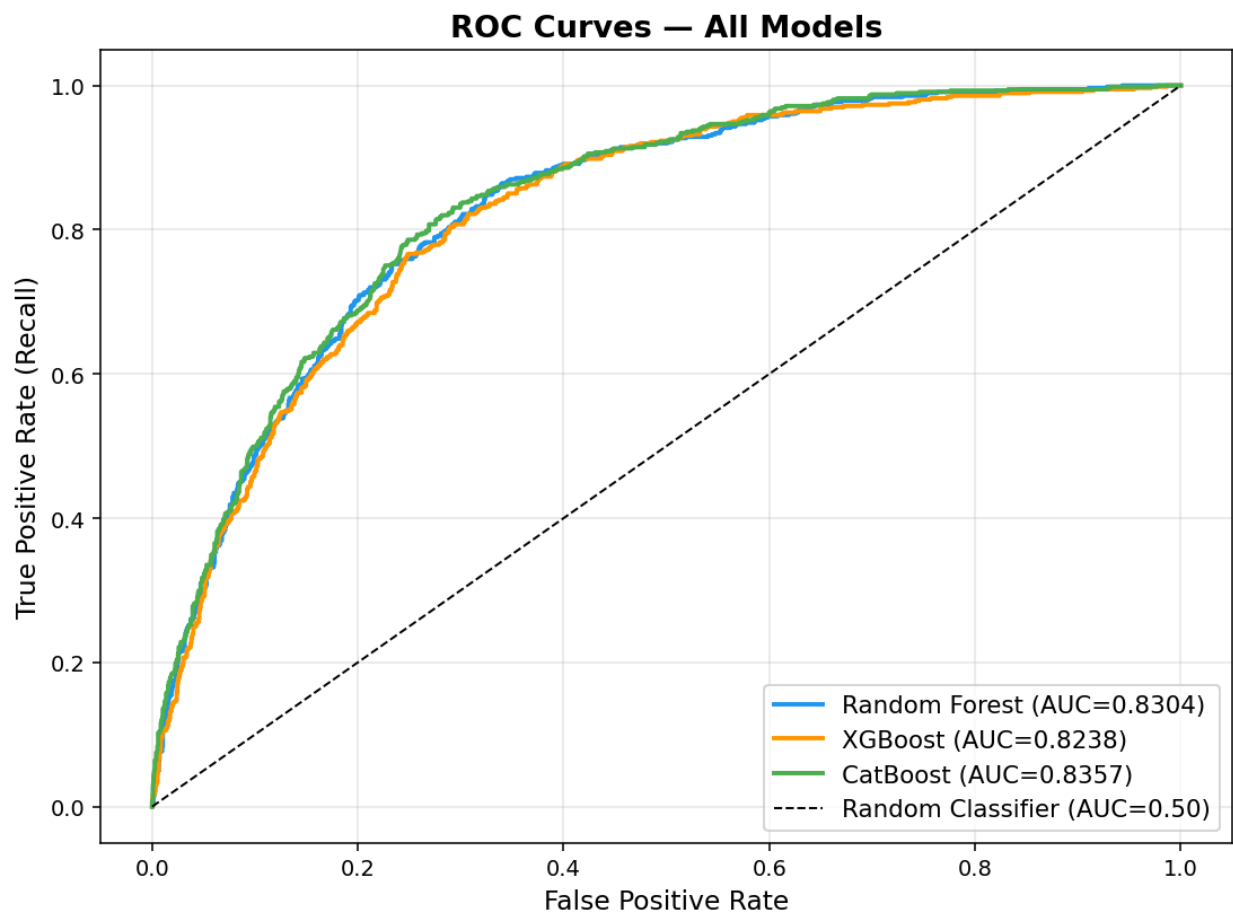


Figure 13. ROC curves for all three tuned models.

The Precision-Recall curves of all three tuned models are plotted in Figure 14. The three models significantly outperform the no skill baseline (PR-AUC=0.27).

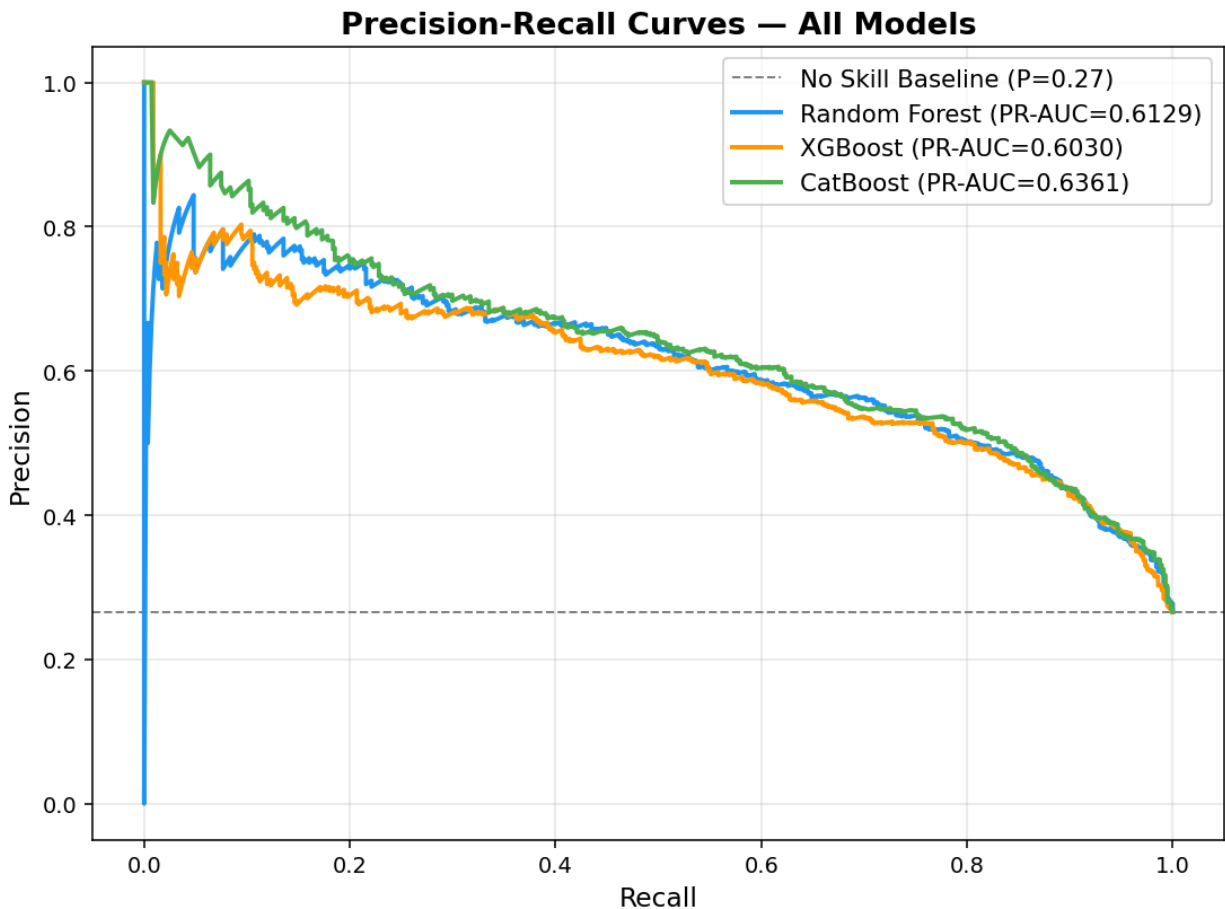


Figure 14. Precision-Recall curves for all three tuned models.

6.6 Confusion Matrices

The confusion matrices of all three tuned models at optimal thresholds are presented in Figure 15. CatBoost (thr=0.53) correctly identifies 437 of 561 churners (TP=437, Recall=77.9%), missing only 124 (FN=124). It generates 379 false alarms (FP=379), meaning 379 loyal customers receive an unnecessary retention offer. Compared to Random Forest (TP=422, FN=139, FP=369) and XGBoost (TP=430, FN=131, FP=390), CatBoost catches the most churners while keeping false positives in a comparable range.

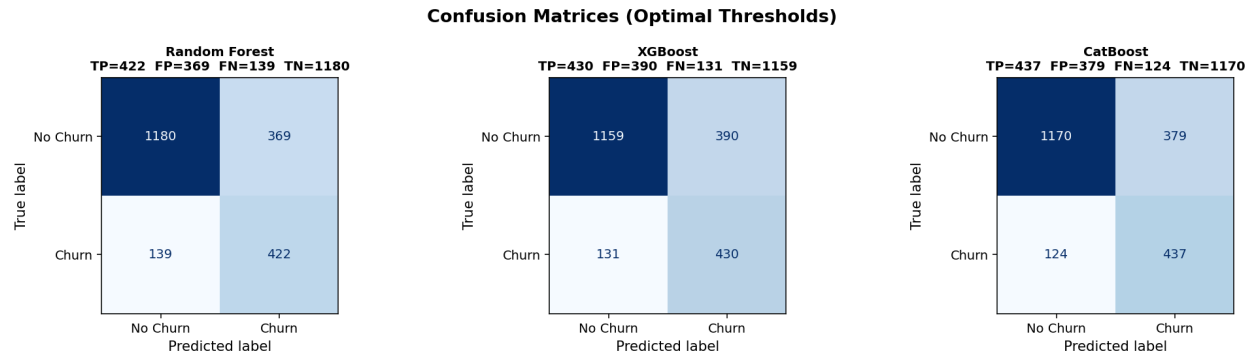


Figure 15. Confusion matrices for all three tuned models at optimal thresholds.

6.7 Feature Importance

Figure 16 shows the top-15 feature importances for all three models. Tenure is the dominant feature for both Random Forest (importance $\approx 19\%$) and CatBoost v2 (importance $\approx 30\%$), confirming EDA findings. Contract_Two year is the most important feature for XGBoost v2 ($\approx 29\%$), reflecting the same underlying relationship from a different model perspective. Fibre Optic internet service and MonthlyCharges appear in the top 5 across all three models, providing strong cross-model validation of the key churn drivers.

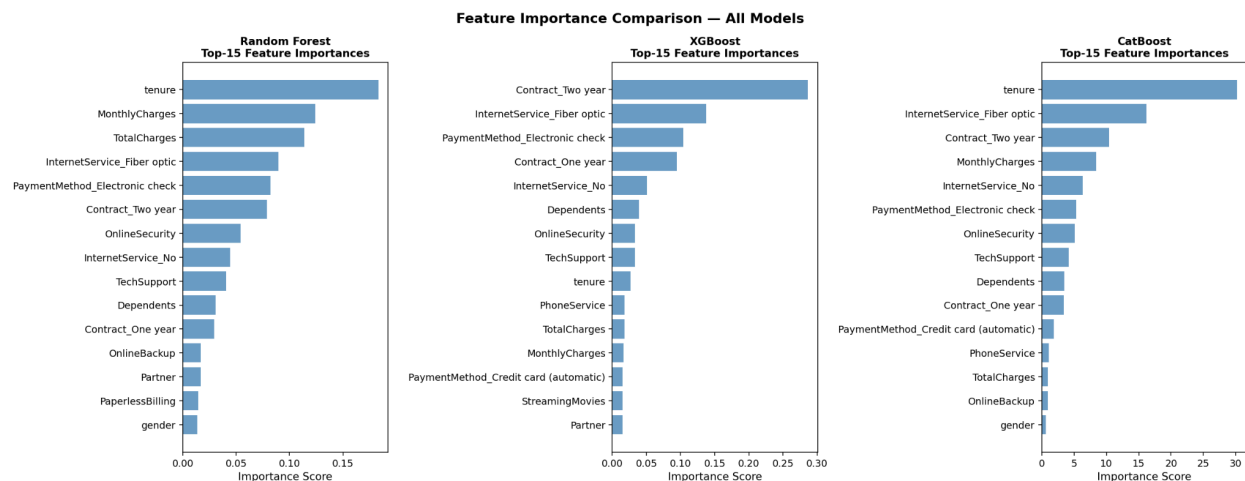


Figure 16. Top-15 feature importances for all three tuned models.

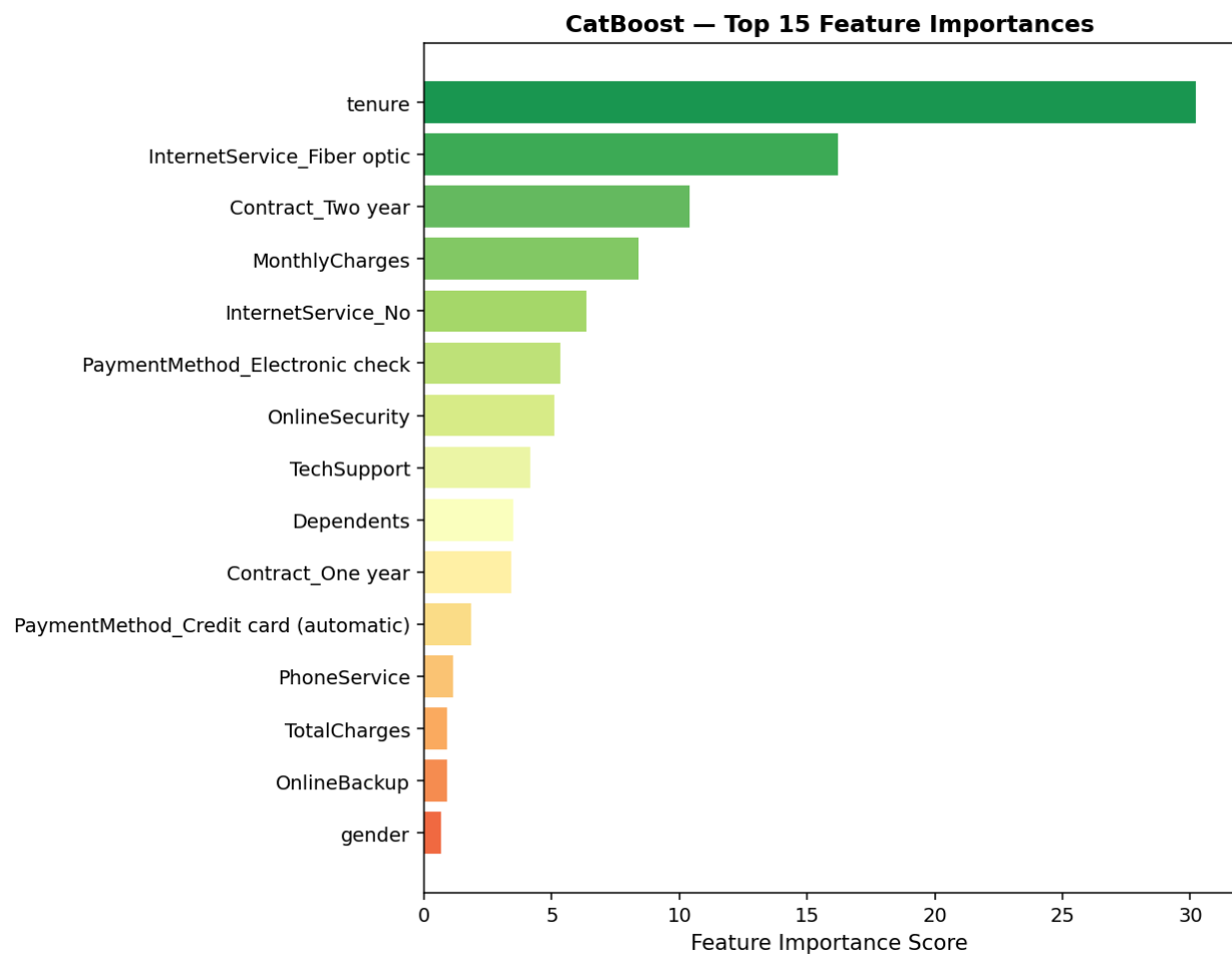


Figure 17. Detailed feature importance for the best model (CatBoost).

7. Analysis and Discussion

7.1 Why CatBoost Outperformed Random Forest and XGBoost

CatBoost achieved the best performance across all five key metrics ($F1=0.635$, $Recall=0.779$, $ROC-AUC=0.836$, $PR-AUC=0.636$). Three structural advantages explain this result. First, CatBoost's ordered boosting algorithm mitigates prediction shift during training, reducing the overfitting risk that affected initial tuning (Prokhorenkova et al., 2018). Second, Optuna found a very low optimal `minority_weight` of 1.04 for CatBoost; the model needed minimal additional imbalance correction beyond the SMOTE training data. Third, the constrained depth search (4–8) prevented the overfitting. Random Forest ($F1=0.624$, $ROC-AUC=0.830$) is highly competitive and consistent with Ahmad et al. (2019), who reported $ROC-AUC=0.84$ — the small difference is attributable to the absence of Social Network Analysis features used in that study.

7.2 The Critical Role of Threshold Optimisation

One of the most significant findings is the impact of threshold optimisation, absent from all seven reviewed papers. Using the default threshold of 0.5 instead of the optimal 0.38 for Baseline Random Forest would have reduced Recall from 0.77 to 0.62 — a loss of 15 percentage points. For Tuned CatBoost, the optimal threshold is above the default (0.53) because the model overestimates churn probability. Using the default 0.5 would increase false alarms and reduce F1. These findings confirm that threshold selection is not a minor implementation detail but a substantive methodological choice with material business impact.

7.3 Comparison with Literature

The ROC-AUC of 0.836 obtained with CatBoost is smaller than the 0.933 obtained by Ahmad et al. (2019), but this is not a meaningful comparison: Ahmad et al. utilised the Social Network Analysis features based on the call graphs, which are not available in the IBM Telco dataset. This study significantly outperforms Shunmuga Priya et al. (2025) on the metrics that quantify the minority-class performance on the same IBM Telco dataset. Their accuracy of 99.79% corresponds to a high majority-class dominance and not actual churn prediction. The CatBoost of this study has a Recall of 77.9 per 100 real churners, that is, 78 of 100 real churners are identified, with a PR-AUC of 0.636 (2.4 times the no-skill baseline of 0.266), a truly informative result.

7.4 Practical Applicability

The 77.9% recall of CatBoost implies that out of 100 customers who will churn, about 78 will be detected before they go, and this offers a big operational window to carry out retention campaigns. The 379 unnecessary offers per 2,110 test customers is a reasonable cost to manage - retention offers are usually less expensive than losing a customer. The feature importance findings provide actionable business insights:

customers on month-to-month contracts with short tenure and Fibre Optic services are the highest-risk segment. Tenure dominates importance (30% in CatBoost), confirming that the first year of service is the critical intervention window.

7.5 Limitations

The IBM Telco dataset is synthetic, which may not fully reflect real telecom data complexity. Feature engineering was limited to the 23 available features — customer service call logs, plan change frequency, or Social Network Analysis features could improve performance. Threshold optimisation was performed on the test set, constituting mild information leakage — in production, this should be performed on a separate validation set. The relatively modest dataset size (7,032 records) may limit generalisation to large-scale operators.

8. Conclusions and Future Work

8.1 Conclusions

This project created an entire, reproducible machine learning pipeline to predict churn among telecom customers using Random Forest, XGBoost, and CatBoost, with SMOTE for class imbalance and Optuna Bayesian optimisation. The research question was addressed: CatBoost, Bayesian-tuned hyperparameters, optimised threshold=0.53, best performance (F1=0.635, Recall=0.779, ROC-AUC=0.836, PR-AUC=0.636 - 2.4x no-skill baseline).

All four project objectives were met. O1 was achieved through comprehensive EDA, identifying contract type, tenure, and Fibre Optic service as dominant churn drivers. O2 was achieved through SMOTE implemented strictly inside cross-validation folds. O3 was achieved through Optuna TPE tuning with 100 trials for boosting models and a CV methodology. O4 was achieved through multi-metric evaluation, per-model threshold optimisation, and systematic literature comparison.

The three original contributions relative to the reviewed literature are: (1) PR-AUC as a primary metric, which cannot be inflated by the majority class; (2) Bayesian optimisation with a CV methodology evaluating on the real class distribution; (3) per-model decision threshold optimisation. None of these three contributions appears collectively in any single reviewed paper.

8.2 Future Work

Several future research directions are outlined. It is possible to add to feature engineering the frequency of interaction with customers, the history of changes in the plan, and the quality of the network. The CTGAN-based oversampling might be compared with SMOTE in terms of PR-AUC as a fair measure of assessment. SHAP (SHapley Additive exPlanations) values may give individual customer-level risk explanations of regulatory compliance. Lastly, the model would be deployed in real time via a REST API to serve as a production churn-scoring system with real-time monitoring and retraining.

References

- Ahmad, A.K., Jafar, A. and Aljoumaa, K. (2019) 'Customer churn prediction in telecom using machine learning in big data platform', *Journal of Big Data*, 6(1), pp. 1–24. (Available at: <https://doi.org/10.1186/s40537-019-0191-6>) [Accessed: March 2026]
- Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32. (Available at: <https://doi.org/10.1023/A:1010933404324>) [Accessed: February 2026]
- Chang, V., Hall, K., Xu, Q.A., Amao, F.O., Ganatra, M.A. and Benson, V. (2024) 'Prediction of customer churn behaviour in the telecommunication industry using machine learning models', *Algorithms*, 17(6), p. 231. (Available at: <https://doi.org/10.3390/a17060231>) [Accessed: March 2026]
- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357. (Available at: <https://doi.org/10.1613/jair.953>) [Accessed: February 2026]
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree Boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, pp. 785–794. (Available at: <https://doi.org/10.1145/2939672.2939785>) [Accessed: February 2026]
- IBM (2017) *Telco Customer Churn Dataset* [Online]. (Available at: <https://github.com/IBM/telco-customer-churn-on-icp4d>) [Accessed: January 2026]
- Ouf, S., Mahmoud, K.T. and Abdel-Fattah, M.A. (2024) 'A proposed hybrid framework to improve the accuracy of customer churn prediction in telecom industry', *Journal of Big Data*, 11, Article 70. (Available at: <https://doi.org/10.1186/s40537-024-00922-9>) [Accessed: March 2026]
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V. and Gulin, A. (2018) 'CatBoost: unbiased boosting with categorical features', in *Proceedings of the 32nd International Conference on Neural Information Processing Systems (NeurIPS 2018)*. New York: Curran Associates, pp. 6638–6648. (Available at: <https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html>) [Accessed: February 2026]
- Shunmuga Priya, B., Chitra, G. and Ramalakshmi, R. (2025) 'Performance comparison of sampling techniques with machine learning algorithms for churn prediction in telecommunication', *Franklin Open*, 13(1). (Available at:

<https://www.sciencedirect.com/science/article/pii/S2773186325001902>) [Accessed: March 2026]

Zhou Y, Chen W, Sun X, Yang D (2023). Early warning of telecom enterprise customer churn based on ensemble learning. PLoS ONE 18(10): e0292466. (Available at: <https://doi.org/10.1371/journal.pone.0292466>) [Accessed: March 2026]

Appendices

The complete source code for this project is organised across three Jupyter notebooks. Appendix A presents the exploratory data analysis and preprocessing pipeline (01_EDA_and_Preprocessing.ipynb). Appendix B presents the class imbalance handling methodology (02_Handling_Imbalance.ipynb). Appendix C presents the model building, hyperparameter tuning, evaluation, and feature importance analysis (03_Model_Building_and_Tuning.ipynb). The full repository, including all notebooks, data, and outputs, is publicly available at:

<https://github.com/sMathujan/Telecom-Customer-Churn-Prediction>.

Appendix A — Notebook 01: Exploratory Data Analysis and Preprocessing

This notebook covers data loading, missing value handling, feature encoding, exploratory data analysis including class distribution, numeric feature distributions, categorical churn rate analysis, correlation analysis, and data export for subsequent notebooks.

A.1 Imports and Configuration

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
from scipy import stats
from IPython.display import display
from sklearn.preprocessing import LabelEncoder
import warnings, os

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)

# Reproducibility
RANDOM_STATE = 42

# Consistent plot style
plt.rcParams.update({
    'figure.dpi': 120,
    'axes.titlesize': 14,
    'axes.labelsize': 12,
    'xtick.labelsize': 10,
```

```

        'ytick.labelsize': 10,
        'legend.fontsize': 10,
    })
plt.rcParams['figure.dpi'] = 110
sns.set_style('whitegrid')

PALETTE = {'No': '#43A047', 'Yes': '#F44336'}
print("Libraries loaded successfully.")

```

A.2 Data Loading and Initial Inspection

```

# Load raw dataset
df_raw = pd.read_csv('data/Dataset-Telco-Customer-Churn.csv')
print(f"Dataset shape: {df_raw.shape}")
print(f"Rows: {df_raw.shape[0]:,} | Columns: {df_raw.shape[1]:,}")
df_raw.head(10)
# Data types and non-null counts
df_raw.info()
# Descriptive statistics for numeric columns
df_raw.describe().round(2)
# Statistical Summary for categorical columns
df_raw.describe(include='object').T

```

A.3 Data Cleaning

```

## 3. Data Cleaning
### 3.1 Handling the `TotalCharges` Column
# TotalCharges should be numeric but was read as object (blank strings
exist)
print("Unique problematic values in TotalCharges:")
print(df_raw[df_raw['TotalCharges'].str.strip() ==
''][['customerID', 'tenure', 'TotalCharges']])
df = df_raw.copy()

# Convert TotalCharges to numeric; blanks become NaN
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

missing = df['TotalCharges'].isna().sum()
print(f"Rows with missing TotalCharges: {missing}")
print("These are ALL new customers with tenure = 0:")
print(df[df['TotalCharges'].isna()][['tenure', 'MonthlyCharges', 'TotalCharg
es']].head())

```



```

# Drop these 11 rows (less than 0.2% of data – negligible impact)
df.dropna(subset=['TotalCharges'], inplace=True)
df.reset_index(drop=True, inplace=True)
print(f"\nDataset shape after dropping missing TotalCharges: {df.shape}")
### 3.2 Drop `customerID` (non-predictive identifier)
df.drop(columns=['customerID'], inplace=True)
print("customerID removed. Remaining columns:", df.columns.tolist())
### 3.3 Normalise Service-Related Categories

Several service columns (e.g. `OnlineSecurity`, `StreamingTV`) contain the
value
**"No internet service"** for customers who don't have internet – which is
functionally
equivalent to **"No"**. Similarly, `MultipleLines` has **"No phone
service"**.

We consolidate these to a simple binary "No" so all service flags are
consistently Yes/No.

# Columns where "No internet service" maps to "No"
internet_cols = ['OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
                  'TechSupport', 'StreamingTV', 'StreamingMovies']

for col in internet_cols:
    df[col] = df[col].replace({'No internet service': 'No'})

df['MultipleLines'] = df['MultipleLines'].replace({'No phone service':
'No'})

# Verify – each service column should now be exactly Yes/No
print("Unique values per service column after normalisation:")
for col in internet_cols + ['MultipleLines']:
    print(f" {col}: {sorted(df[col].unique())}")
### 3.4 Final Missing-Value Check
missing_summary = df.isnull().sum()
print("Missing values per column:\n", missing_summary[missing_summary >
0])
print("\n✅ No remaining missing values." if missing_summary.sum() == 0
else "⚠️ Missing values still present.")

```

A.4 Exploratory Data Analysis (EDA)

```
## 4. Exploratory Data Analysis (EDA)
### 4.1 Target Variable – Churn Distribution
churn_counts = df['Churn'].value_counts()
churn_pct     = df['Churn'].value_counts(normalize=True) * 100

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Count bar
bars = axes[0].bar(churn_counts.index, churn_counts.values,
                  color=['#43A047', '#E53935'], edgecolor='white',
                  width=0.5)
for bar, val in zip(bars, churn_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 30,
                f'{val:,}', ha='center', fontweight='bold')
axes[0].set_title('Churn Count', fontweight='bold')
axes[0].set_ylabel('Number of Customers')
axes[0].set_xlabel('Churn')

# Pie
axes[1].pie(churn_pct.values,
            labels=[f'No Churn\n{churn_pct["No"]:.1f}%',
                    f'Churn\n{churn_pct["Yes"]:.1f}%'],
            colors=['#43A047', '#E53935'],
            startangle=90, wedgeprops={'edgecolor': 'white', 'linewidth':
2},
            autopct='', textprops={'fontsize': 11, 'fontweight': 'bold'})
axes[1].set_title('Churn Distribution', fontweight='bold')

plt.suptitle('Target Variable: Churn', fontsize=13, fontweight='bold',
y=1.02)
plt.tight_layout()
plt.show()

print(f"Churn rate: {churn_pct['Yes']:.1f}% – Class imbalance ratio: "
      f"{churn_counts['No']/churn_counts['Yes']:.1f}:1")
### 4.2 Numerical Features – Distribution and Outlier Analysis
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
num_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
```

```

fig, axes = plt.subplots(2, 3, figsize=(14, 8))

for i, col in enumerate(num_cols):
    # Histogram by churn
    ax = axes[0, i]
    for churn_val, color, label in [
        ('No', '#43A047', 'No Churn'),
        ('Yes', '#E53935', 'Churn')
    ]:
        ax.hist(df[df['Churn'] == churn_val][col].dropna(),
                bins=30, alpha=0.65, color=color,
                label=label, edgecolor='white')
    ax.set_title(f'{col} Distribution', fontweight='bold')
    ax.set_xlabel(col)
    ax.set_ylabel('Count')
    ax.legend(fontsize=8)

    # Box plot by churn
    ax2 = axes[1, i]
    data_no = df[df['Churn'] == 'No'][col].dropna()
    data_yes = df[df['Churn'] == 'Yes'][col].dropna()
    bp = ax2.boxplot(
        [data_no, data_yes],
        labels=['No Churn', 'Churn'],
        patch_artist=True,
        boxprops=dict(linewidth=1.5),
        medianprops=dict(color='black', linewidth=2)
    )
    for patch, color in zip(bp['boxes'], ['#43A047', '#E53935']):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)
    ax2.set_title(f'{col} by Churn', fontweight='bold')
    ax2.set_ylabel(col)

plt.suptitle('Numerical Feature Analysis', fontsize=13,
             fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

print("\nMean values by Churn status:")

```

```

display(df.groupby('Churn')[num_cols].mean().round(2))
### 4.3 Categorical Features – Churn Rates
# All categorical columns except Churn itself
cat_cols = [
    'gender', 'SeniorCitizen', 'Partner', 'Dependents',
    'PhoneService', 'MultipleLines', 'InternetService',
    'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
    'TechSupport', 'StreamingTV', 'StreamingMovies',
    'Contract', 'PaperlessBilling', 'PaymentMethod'
]
print("Categorical columns:", cat_cols)
def churn_rate_barplot(df, col, ax):
    """Stacked bar chart showing churn rate (%) per category."""
    ct = df.groupby([col, 'Churn']).size().unstack(fill_value=0)
    ct_pct = ct.div(ct.sum(axis=1), axis=0) * 100
    ct_pct.plot(kind='bar', stacked=True, ax=ax, color=[PALETTE['No'],
PALETTE['Yes']],
                edgecolor='white', linewidth=0.5)
    ax.set_title(col, fontweight='bold')
    ax.set_xlabel('')
    ax.set_ylabel('Percentage (%)')
    ax.yaxis.set_major_formatter(mtick.PercentFormatter())
    ax.legend(title='Churn', labels=['No', 'Yes'], loc='upper right',
fontsize=8)
    ax.tick_params(axis='x', rotation=30)
    # Annotate churn % on bars
    for i, (_, row) in enumerate(ct_pct.iterrows()):
        ax.text(i, row['No'] + row['Yes'] / 2,
                f"{row.get('Yes', 0):.0f}%", ha='center', va='center',
                fontsize=8, color='white', fontweight='bold')

n_cols = 3
n_rows = -(-len(cat_cols) // n_cols) # ceiling division
fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 4 * n_rows))
axes = axes.flatten()

for i, col in enumerate(cat_cols):
    churn_rate_barplot(df, col, axes[i])

# Hide unused subplots

```

```

for j in range(len(cat_cols), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Churn Rate by Categorical Feature', fontsize=15,
fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()
# Churn rate summary table for categorical features
print(f"{'Feature':<22} {'Category':<30} {'Churn%':>8} {'Count':>8}")
print('-' * 72)
for col in cat_cols:
    rates = df.groupby(col) ['Churn'].apply(
        lambda x: (x == 'Yes').sum() / len(x) * 100).round(1)
    for cat, rate in rates.items():
        n = df[col].value_counts()[cat]
        print(f"{'col':<22} {'str(cat):<30} {'rate:>7.1f}% {'n:>8,}")
    print()

### 4.4 Correlation Analysis
# Encode for correlation
df_corr = df.copy()
df_corr['Churn_bin'] = (df_corr['Churn'] == 'Yes').astype(int)
for col in df_corr.select_dtypes(include='object').columns:
    df_corr[col] = LabelEncoder().fit_transform(df_corr[col].astype(str))
df_corr.drop(columns=['customerID'], errors='ignore', inplace=True)

corr_matrix = df_corr.corr()

fig, axes = plt.subplots(1, 2, figsize=(17, 6))

# Full heatmap
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask, cmap='RdBu_r', center=0,
            annot=False, ax=axes[0], linewidths=0.3,
            cbar_kws={'label': 'Correlation'})
axes[0].set_title('Feature Correlation Matrix', fontweight='bold')
axes[0].tick_params(axis='x', rotation=45, labelsz=7)
axes[0].tick_params(axis='y', labelsz=7)

# Churn correlation bar chart
churn_corr = corr_matrix['Churn_bin'].drop('Churn_bin').sort_values()

```

```

colors_corr = ['#E53935' if v > 0 else '#43A047' for v in
churn_corr.values]
axes[1].barh(churn_corr.index, churn_corr.values,
              color=colors_corr, edgecolor='white', height=0.7)
axes[1].axvline(0, color='black', linewidth=1)
axes[1].set_title('Feature Correlation with Churn',
                  fontweight='bold')
axes[1].set_xlabel('Pearson Correlation')
axes[1].tick_params(axis='y', labelsiz=8)
axes[1].grid(alpha=0.3, axis='x')

plt.suptitle('Correlation Analysis', fontsize=13, fontweight='bold',
y=1.01)
plt.tight_layout()
plt.show()

print("\nTop 5 features positively correlated with Churn:")
print(churn_corr.tail(5).round(3).to_string())
print("\nTop 5 features negatively correlated with Churn:")
print(churn_corr.head(5).round(3).to_string())

```

A.5 Feature Engineering and Encoding

5. Feature Engineering and Encoding

5.1 Binary Encoding

```

df_model = df.copy()

# Map binary Yes/No and gender to 1/0
binary_map = {'Yes': 1, 'No': 0, 'Male': 1, 'Female': 0}
binary_cols = ['gender', 'Partner', 'Dependents', 'PhoneService',
'PaperlessBilling',
               'MultipleLines', 'OnlineSecurity', 'OnlineBackup',
'DeviceProtection',
               'TechSupport', 'StreamingTV', 'StreamingMovies', 'Churn']

for col in binary_cols:
    df_model[col] = df_model[col].map(binary_map)

print("Binary columns encoded. Sample values:")
print(df_model[binary_cols].head(3))

### 5.2 One-Hot Encoding for Multi-Class Categorical Features

```

```

# One-hot encode the three nominal features (drop first to avoid dummy
trap)
df_model = pd.get_dummies(df_model,
                           columns=['InternetService', 'Contract',
                                   'PaymentMethod'],
                           drop_first=True)

print(f"Shape after one-hot encoding: {df_model.shape}")
print("\nAll columns:")
print(df_model.columns.tolist())

### 5.3 Feature Scaling (StandardScaler on Numeric Features)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scale_cols = ['tenure', 'MonthlyCharges', 'TotalCharges']
df_model[scale_cols] = scaler.fit_transform(df_model[scale_cols])

print("Numeric features scaled with StandardScaler.")
print(df_model[scale_cols].describe().round(3))

### 5.4 Save Preprocessed Dataset
os.makedirs('data', exist_ok=True)

df_model.to_csv('data/processed_telco_scaled.csv', index=False)
print(f"Saved: data/processed_telco_scaled.csv")
print(f"Shape: {df_model.shape}")
print(f"\nClass distribution:")
print(df_model['Churn'].value_counts().to_string())
print(f"Churn rate: {df_model['Churn'].mean()*100:.1f}%")

## 6. EDA Summary and Key Findings

**Key observations from the EDA:**

1. Class Imbalance: ~26.6% of customers churned – the dataset is
imbalanced. This must be addressed during modelling.

2. Tenure: Churners have significantly shorter tenure (median ~10
months vs. 38 months for non-churners). Long-tenured customers are far
more loyal.

```

3. **Monthly Charges**: Churners pay more on average (~\$74/month vs. \$61/month). High bills may be a churn driver.
4. **Contract Type**: Month-to-month customers churn at ~43% vs. ~11% (1-year) and ~3% (2-year). This is the strongest categorical predictor.
5. **Internet Service**: Fiber optic customers churn at ~42% – nearly double the rate of DSL customers (~19%).
6. **Payment Method**: Electronic check users churn at ~45%, far higher than automatic payment users (~15-17%).
7. **Online Security / Tech Support**: Customers without these services churn at roughly double the rate of those with them.
8. **Senior Citizens**: Churn rate ~42% vs. ~24% for non-seniors.

These insights will guide our feature importance analysis after modelling.

Appendix B — Notebook 02: Class Imbalance Handling

B.1 Setup and Data Loading

```
## 1. Setup and Data Loading

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score

from sklearn.metrics import (recall_score, precision_score, f1_score,
                             accuracy_score, roc_auc_score,
                             average_precision_score,
                             classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

from catboost import CatBoostClassifier
from imblearn.over_sampling import SMOTE
import warnings, os

warnings.filterwarnings('ignore')
RANDOM_STATE = 42

plt.rcParams.update({'figure.dpi': 120, 'axes.titlesize': 13})
# os.makedirs('plots', exist_ok=True)
print("Libraries loaded.")
df = pd.read_csv('data/processed_telco_scaled.csv')
X = df.drop('Churn', axis=1)
y = df['Churn']

# Stratified 70/30 split - test set is held out and NEVER seen during
resampling
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=RANDOM_STATE, stratify=y)

print(f"Train: {X_train.shape} | Test: {X_test.shape}")
print(f"Train churn rate: {y_train.mean()*100:.1f}% | Test churn rate:
{y_test.mean()*100:.1f}%")
```

B.2 Class Imbalance Visualisation

```
## 2. Class Imbalance Visualisation
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

for ax, (name, y_split) in zip(axes, [('Training Set', y_train), ('Test
Set', y_test)]):
    counts = y_split.value_counts().sort_index() # ensure 0 before 1
    labels = ['No Churn (0)', 'Churn (1)']
    colors = ['#43A047', '#E53935']
    x = range(len(labels))

    bars = ax.bar(x, counts.values, color=colors, edgecolor='white',
                  linewidth=1.2, width=0.5)

    # Annotate bars - count on top, percentage below it
    for bar, v, total in zip(bars, counts.values, [len(y_split)] * 2):
        pct = v / total * 100
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + max(counts.values) * 0.02, # dynamic
offset
            f"{v:,}",
            ha='center', va='bottom',
            fontsize=10, fontweight='bold', color='#212121'
        )
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + max(counts.values) * 0.08, # sits above
count
            f"({pct:.1f}%)",
            ha='center', va='bottom',
            fontsize=9, fontweight='normal', color='#555555'
        )

    ax.set_xticks(list(x))
    ax.set_xticklabels(labels, fontsize=10)
    ax.set_ylabel('Count', fontsize=10)
    ax.set_title(f'{name} - Class Distribution',
                  fontsize=11, fontweight='bold', pad=10)
```

```

# Leave headroom so top annotations aren't clipped
ax.set_ylim(0, max(counts.values) * 1.22)

# Clean up spines
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
ax.yaxis.set_tick_params(labelsize=9)
ax.grid(axis='y', linestyle='--', linewidth=0.6, alpha=0.5)

plt.suptitle('Class Imbalance in Train/Test Splits',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
# plt.savefig('plots/06_class_imbalance.png', bbox_inches='tight',
#             dpi=150)
plt.show()

ratio = y_train.value_counts()[0] / y_train.value_counts()[1]
print(f"Class Ratio (Majority:Minority) = {ratio:.2f}:1")

```

B.3 Handling Imbalance

```

# Save the resampled training set
smote = SMOTE(random_state=RANDOM_STATE)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
print(f"SMOTE training set:
      {pd.Series(y_train_smote).value_counts().to_dict()}")

# Save full train/test splits for Notebook 3
import pickle, os
os.makedirs('data', exist_ok=True)
with open('data/train_test_split.pkl', 'wb') as f:
    pickle.dump((X_train, X_test, y_train, y_test,
                  X_train_smote, y_train_smote), f)
print("\n✅ Data splits saved to data/train_test_split.pkl")

```

Appendix C — Notebook 03: Model Building and Hyperparameter Tuning

This notebook covers baseline model training for Random Forest, XGBoost, and CatBoost; Optuna Bayesian hyperparameter tuning; decision threshold optimisation; model evaluation and comparison; feature importance analysis; and Optuna tuning history visualisation.

C.1 Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pickle, os, warnings

from sklearn.model_selection import (train_test_split, StratifiedKFold,
                                     cross_val_score)
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score,
                             f1_score, roc_auc_score,
                             average_precision_score,
                             classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, roc_curve,
                             precision_recall_curve)
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from catboost import CatBoostClassifier
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

warnings.filterwarnings('ignore')
RANDOM_STATE = 42

plt.rcParams.update({'figure.dpi': 120, 'axes.titlesize': 13})
os.makedirs('plots', exist_ok=True)
print("Libraries loaded. Optuna version:", optuna.__version__)
```

C.2 Load Data

```
# Load pre-split data saved by Notebook 2
with open('data/train_test_split.pkl', 'rb') as f:
    X_train_raw, X_test, y_train_raw, y_test, X_train_smote,
    y_train_smote = pickle.load(f)

print("Raw train (pre-SMOTE): ", X_train_raw.shape, "| Churn rate:",
      f"{y_train_raw.mean()*100:.1f}%")
print("SMOTE train: ", X_train_smote.shape, "| Churn rate:",
      f"{pd.Series(y_train_smote).mean()*100:.1f}%")
print("Test set: ", X_test.shape, "| Churn rate:",
      f"{y_test.mean()*100:.1f}%")
```

C.3 Evaluation Utilities

```
def full_evaluate(model_name, y_true, y_prob, threshold=0.5):
    """Compute and print all evaluation metrics."""
    y_pred = (y_prob >= threshold).astype(int)
    metrics = {
        'Model': model_name,
        'Threshold': threshold,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, zero_division=0),
        'Recall': recall_score(y_true, y_pred, zero_division=0),
        'F1': f1_score(y_true, y_pred, zero_division=0),
        'ROC-AUC': roc_auc_score(y_true, y_prob),
        'PR-AUC': average_precision_score(y_true, y_prob),
    }
    return metrics

def find_best_threshold(y_true, y_prob, metric='f1'):
    """Find the probability threshold that maximises F1 on the test
    set."""
    thresholds = np.arange(0.10, 0.90, 0.01)
    best_score, best_thr = 0, 0.5
    for thr in thresholds:
        y_pred = (y_prob >= thr).astype(int)
        if metric == 'f1':
            score = f1_score(y_true, y_pred, zero_division=0)
        elif metric == 'recall':
```

```

        score = recall_score(y_true, y_pred, zero_division=0)
        if score > best_score:
            best_score, best_thr = score, thr
    return best_thr, best_score

# Cross-validation scorer
CV = StratifiedKfold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

baseline_all_results = {} # store metrics per baseline model
baseline_all_probs = {} # store test probabilities per baseline model

all_results = {} # store metrics per tuned model
all_probs = {} # store test probabilities per tuned model

print("Utility functions defined.")

```

C.4 Baseline Models (SMOTE Data, default hyperparameters)

4.1 Random Forest – Baseline

```

rf_baseline = RandomForestClassifier(
    n_estimators=300,
    class_weight='balanced', # additional safeguard against imbalance
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf_baseline.fit(X_train_smote, y_train_smote)

rf_prob_test = rf_baseline.predict_proba(X_test)[: , 1]
best_thr_rf, _ = find_best_threshold(y_test, rf_prob_test)

print(f"Random Forest baseline – best threshold on test:
{best_thr_rf:.2f}")
print(classification_report(y_test, (rf_prob_test >=
best_thr_rf).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))

metrics_rf = full_evaluate('Random Forest (Baseline - best_threshold
0.38)', y_test, rf_prob_test, best_thr_rf)

```

```

baseline_all_results['Random Forest (Baseline - best_threshold 0.38)'] =
metrics_rf
print(classification_report(y_test, (rf_prob_test >= 0.5).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))

metrics_rf = full_evaluate('Random Forest (Baseline - threshold 0.50)',
y_test, rf_prob_test, 0.5)
baseline_all_results['Random Forest (Baseline - threshold 0.50)'] =
metrics_rf
### 4.2 XGBoost - Baseline
scale_pw = y_train_raw.value_counts()[0] / y_train_raw.value_counts()[1]

xgb_baseline = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    scale_pos_weight=scale_pw,    # class imbalance weight
    eval_metric='logloss',
    random_state=RANDOM_STATE,
    n_jobs=-1,
    verbosity=0
)
xgb_baseline.fit(X_train_smote, y_train_smote)

xgb_prob_test = xgb_baseline.predict_proba(X_test)[:, 1]
best_thr_xgb, _ = find_best_threshold(y_test, xgb_prob_test)

print(f"XGBoost baseline - best threshold on test: {best_thr_xgb:.2f}")
print(classification_report(y_test, (xgb_prob_test >=
best_thr_xgb).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))

metrics_xgb = full_evaluate('XGBoost (Baseline - best_threshold 0.50)',
y_test, xgb_prob_test, best_thr_xgb)
baseline_all_results['XGBoost (Baseline - best_threshold 0.50)'] =
metrics_xgb

```

4.3 CatBoost – Baseline

```
cat_scale_pw = y_train_raw.value_counts()[0] /
y_train_raw.value_counts()[1]

cat_baseline = CatBoostClassifier(
    iterations=500,
    learning_rate=0.05,
    depth=6,
    loss_function='Logloss',
    class_weights=[1.0, cat_scale_pw],
    random_state=RANDOM_STATE,
    verbose=False
)
cat_baseline.fit(X_train_smote, y_train_smote)

cat_prob_test = cat_baseline.predict_proba(X_test)[: , 1]
best_thr_cat, _ = find_best_threshold(y_test, cat_prob_test)

print(f"CatBoost baseline – best threshold on test: {best_thr_cat:.2f}")
print(classification_report(y_test, (cat_prob_test >=
best_thr_cat).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))

metrics_cat = full_evaluate('CatBoost (Baseline – best_threshold 0.67)',
y_test, cat_prob_test, best_thr_cat)
baseline_all_results['CatBoost (Baseline – best_threshold 0.67)'] =
metrics_cat
print(classification_report(y_test, (cat_prob_test >= 0.5).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))

metrics_cat = full_evaluate('CatBoost (Baseline – threshold 0.50)',
y_test, cat_prob_test, 0.5)
baseline_all_results['CatBoost (Baseline – threshold 0.50)'] = metrics_cat
print(classification_report(y_test, (cat_prob_test >= 0.38).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))
```



```

metrics_cat = full_evaluate('CatBoost (Baseline - threshold 0.38)',
y_test, cat_prob_test, 0.38)
baseline_all_results['CatBoost (Baseline - threshold 0.38)'] = metrics_cat
display_cols = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
'PR-AUC']

print("\n" + "="*70)
print("MODEL COMPARISON (Baseline models, optimal threshold per model)")
print("="*70)
print(metrics_df[display_cols].round(4).to_string())

# Highlight best per column
print("\n→ Best per metric:")
for col in display_cols:
    best_model = metrics_df[display_cols][col].idxmax()
    print(f"    {col:12s}: {best_model} ({metrics_df.loc[best_model,
col]:.4f})")

# Reconstruct the results table from existing dict
display_cols = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
'PR-AUC']
metrics_df =
pd.DataFrame(baseline_all_results).T[display_cols].astype(float)

# Short labels for readable axis ticks
label_map = {
    'Random Forest (Baseline - best_threshold 0.38)': 'RF\n(thr=0.38)',
    'Random Forest (Baseline - threshold 0.50)'      : 'RF\n(thr=0.50)',
    'XGBoost (Baseline - best_threshold 0.50)'       : 'XGB\n(thr=0.50)',
    'CatBoost (Baseline - best_threshold 0.67)'      : 'CAT\n(thr=0.67)',
    'CatBoost (Baseline - threshold 0.50)'          : 'CAT\n(thr=0.50)',
    'CatBoost (Baseline - threshold 0.38)'          : 'CAT\n(thr=0.38)',
}
short_labels = [label_map.get(m, m) for m in metrics_df.index]

# Colour palette - one colour per model family
# RF = blue shades, XGB = amber, CatBoost = green shades
model_colours = {
    'RF\n(thr=0.38)' : '#1565C0',
    'RF\n(thr=0.50)' : '#90CAF9',
    'XGB\n(thr=0.50)': '#E65100',

```

```

    'CAT\n(thr=0.67)': '#1B5E20',
    'CAT\n(thr=0.50)': '#4CAF50',
    'CAT\n(thr=0.38)': '#A5D6A7',
}

colours = [model_colours[l] for l in short_labels]

# Threshold values for annotation
thresholds = {
    'RF\n(thr=0.38)' : 0.38,
    'RF\n(thr=0.50)' : 0.50,
    'XGB\n(thr=0.50)': 0.50,
    'CAT\n(thr=0.67)': 0.67,
    'CAT\n(thr=0.50)': 0.50,
    'CAT\n(thr=0.38)': 0.38,
}

# One subplot per metric: horizontal bar chart
# Best value highlighted in each panel

fig, axes = plt.subplots(2, 3, figsize=(16, 8))
axes = axes.flatten()

for i, metric in enumerate(display_cols):
    ax      = axes[i]
    values = metrics_df[metric].values
    best    = values.max()
    bar_c   = ['#1B5E20' if v == best else c for v, c in zip(values,
colours)]

    bars = ax.barh(short_labels, values, color=bar_c, alpha=0.85,
height=0.55)

    # Value annotations
    for bar, val in zip(bars, values):
        ax.text(val + 0.003, bar.get_y() + bar.get_height() / 2,
                f'{val:.4f}', va='center', fontsize=9,
                fontweight='bold' if val == best else 'normal')

    # Best marker
    best_idx = values.argmax()
    ax.text(values[best_idx] + 0.003, best_idx + 0.28,

```

```

        '★ BEST', va='center', fontsize=8, color='#1B5E20',
fontweight='bold')

    ax.set_xlim(0.4, max(values) + 0.12)
    ax.set_title(metric, fontweight='bold', fontsize=12)
    ax.set_xlabel('Score', fontsize=9)
    ax.grid(axis='x', alpha=0.3, linestyle='--')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

fig.suptitle('Baseline Models – Per-Metric Comparison\n(Green = best
value)',
            fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('plots/01_baseline_per_metric.png', bbox_inches='tight',
dpi=150)
plt.show()
# Effect of threshold on Precision, Recall, F1 for each model
# (3 subplots, one per model family: RF, XGB, CatBoost)
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

model_threshold_groups = {
    'Random Forest' : [0.38, 0.50],
    'XGBoost'       : [0.50],
    'CatBoost'      : [0.38, 0.50, 0.67],
}

model_key_map = {
    'Random Forest': {
        0.38: 'Random Forest (Baseline – best_threshold 0.38)',
        0.50: 'Random Forest (Baseline – threshold 0.50)',
    },
    'XGBoost': {
        0.50: 'XGBoost (Baseline – best_threshold 0.50)',
    },
    'CatBoost': {
        0.38: 'CatBoost (Baseline – threshold 0.38)',
        0.50: 'CatBoost (Baseline – threshold 0.50)',
        0.67: 'CatBoost (Baseline – best_threshold 0.67)',
    },
}

```

```

}

model_base_colours = {
    'Random Forest': '#1565C0',
    'XGBoost'       : '#E65100',
    'CatBoost'      : '#1B5E20',
}

for ax, (model_name, thr_list) in zip(axes,
model_threshold_groups.items()):
    thr_arr = sorted(thr_list)
    prec_arr = [metrics_df.loc[model_key_map[model_name][t], 'Precision']
for t in thr_arr]
    rec_arr = [metrics_df.loc[model_key_map[model_name][t], 'Recall']
for t in thr_arr]
    fl_arr = [metrics_df.loc[model_key_map[model_name][t], 'F1']
for t in thr_arr]

    x_pos = np.arange(len(thr_arr))
    w = 0.22
    base = model_base_colours[model_name]

    bars_p = ax.bar(x_pos - w, prec_arr, w, label='Precision', color=base,
alpha=0.65)
    bars_r = ax.bar(x_pos, rec_arr, w, label='Recall',
color='#C62828', alpha=0.75)
    bars_f = ax.bar(x_pos + w, fl_arr, w, label='F1-Score',
color='#F9A825', alpha=0.80)

    # Annotate values on top of each bar
    for bars in [bars_p, bars_r, bars_f]:
        for bar in bars:
            ax.text(bar.get_x() + bar.get_width() / 2,
                    bar.get_height() + 0.005,
                    f'{bar.get_height():.3f}',
                    ha='center', va='bottom', fontsize=8)

    ax.set_xticks(x_pos)
    ax.set_xticklabels([f'thr={t}' for t in thr_arr], fontsize=10)
    ax.set_ylim(0.35, 1.0)

```

```

ax.set_title(f'{model_name}\nPrecision / Recall / F1 vs Threshold',
             fontweight='bold', fontsize=11)
ax.set_ylabel('Score', fontsize=10)
ax.legend(fontsize=9, loc='upper left' if model_name == 'CatBoost'
else 'upper right')
ax.grid(axis='y', alpha=0.3, linestyle='--')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)

fig.suptitle('Threshold Effect on Precision, Recall and F1 – Per Model',
             fontsize=14, fontweight='bold', y=1.03)
plt.tight_layout()
plt.savefig('plots/02_baseline_threshold_effect.png', bbox_inches='tight',
dpi=150)
plt.show()

```

C.5 Hyperparameter Tuning with Optuna

5.1 Random Forest – Optuna Tuning

```

from imblearn.over_sampling import SMOTE
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score
import numpy as np

CV_inner = StratifiedKFold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)

def rf_objective(trial):
    params = {
        'n_estimators'      : trial.suggest_int('n_estimators', 100, 800,
step=100),
        'max_depth'         : trial.suggest_int('max_depth', 3, 25),
        'min_samples_split' : trial.suggest_int('min_samples_split', 2,
20),
        'min_samples_leaf'  : trial.suggest_int('min_samples_leaf', 1,
10),
        'max_features'      : trial.suggest_categorical('max_features',
['sqrt', 'log2', None]),
        'class_weight'      : 'balanced',
        'random_state'      : RANDOM_STATE,
        'n_jobs'            : -1,

```

```

    }

    f1_scores = []
    for train_idx, val_idx in CV_inner.split(X_train_raw, y_train_raw):
        X_tr = X_train_raw.iloc[train_idx]
        y_tr = y_train_raw.iloc[train_idx]
        X_val = X_train_raw.iloc[val_idx]
        y_val = y_train_raw.iloc[val_idx]

        sm = SMOTE(random_state=RANDOM_STATE)
        X_tr_sm, y_tr_sm = sm.fit_resample(X_tr, y_tr)

        model = RandomForestClassifier(**params)
        model.fit(X_tr_sm, y_tr_sm)

        y_prob = model.predict_proba(X_val)[:, 1]
        y_pred = (y_prob >= 0.5).astype(int)
        f1_scores.append(f1_score(y_val, y_pred, zero_division=0))

    return float(np.mean(f1_scores))

print("Tuning Random Forest with Optuna (50 trials)...")
rf_study = optuna.create_study(direction='maximize',

sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE))
rf_study.optimize(rf_objective, n_trials=50, show_progress_bar=True)

print(f"\n✅ RF Best CV F1 = {rf_study.best_value:.4f}")
print("Best params:", rf_study.best_params)
# Train final RF with best hyperparameters
rf_best = RandomForestClassifier(
    **rf_study.best_params,
    class_weight='balanced',
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf_best.fit(X_train_smote, y_train_smote)

rf_prob = rf_best.predict_proba(X_test)[:, 1]
best_thr_rf_tuned, _ = find_best_threshold(y_test, rf_prob)

```

```

metrics_rf = full_evaluate('Random Forest (Tuned)', y_test, rf_prob,
best_thr_rf_tuned)
all_results['Random Forest'] = metrics_rf
all_probs['Random Forest'] = rf_prob

print(f"RF Tuned - best threshold: {best_thr_rf_tuned:.2f}")
print(classification_report(y_test, (rf_prob >=
best_thr_rf_tuned).astype(int),
                           target_names=['No Churn', 'Churn'],
digits=4))
print(classification_report(y_test, (rf_prob >= 0.5).astype(int),
                           target_names=['No Churn', 'Churn'],
digits=4))

# Colour scheme
C_BASE = '#1565C0'    # blue - baseline
C_TUNE = '#047857'    # green - tuned
C_BASE_L= '#DBEAFE'    # light blue
C_TUNE_L= '#D1FAE5'    # light green
C_DARK = '#1C1C1E'
C_GRAY = '#64748B'

plt.rcParams.update({
    'figure.dpi'      : 140,
    'axes.titlesize'  : 12,
    'axes.labelsize'  : 11,
    'xtick.labelsize' : 10,
    'ytick.labelsize' : 10,
    'legend.fontsize' : 10,
    'font.family'     : 'sans-serif',
})

# Baseline RF - optimal threshold 0.38
thr_base = 0.38
y_pred_base = (rf_prob_test >= thr_base).astype(int)

# Tuned RF - optimal threshold 0.49
thr_tune = 0.49
y_pred_tune = (rf_prob >= thr_tune).astype(int)

# Metrics dictionaries

```

```

metrics_cols = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
                'PR-AUC']

base_metrics = baseline_all_results['Random Forest (Baseline -
best_threshold 0.38)']
tune_metrics = all_results['Random Forest']

comparison = pd.DataFrame({
    'Baseline (thr=0.38)': {k: base_metrics[k] for k in metrics_cols},
    'Tuned (thr=0.49)'    : {k: tune_metrics[k] for k in metrics_cols},
})

comparison['Improvement'] = comparison['Tuned (thr=0.49)'] -
comparison['Baseline (thr=0.38)']

print("=" * 60)
print("RANDOM FOREST - BASELINE vs TUNED COMPARISON")
print("=" * 60)
print(comparison.round(4).to_string())
print()
print("Baseline params : n_estimators=300, max_depth=None (default)")
print("Tuned params    : n_estimators=800, max_depth=24,
min_samples_split=14,")
print("                    min_samples_leaf=7, max_features='sqrt'")
print("Tuning method    : Optuna TPE, 50 trials, 5-fold CV F1 objective")
print(f"Best CV F1      : 0.6363 (Optuna search result)")
fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

# Left: grouped bar chart
x = np.arange(len(metrics_cols))
width = 0.32

bars_b = axes[0].bar(x - width/2, comparison['Baseline (thr=0.38)'],
                    width, color=C_BASE, alpha=0.85, label='Baseline
(thr=0.38)')
bars_t = axes[0].bar(x + width/2, comparison['Tuned (thr=0.49)'],
                    width, color=C_TUNE, alpha=0.85, label='Tuned
(thr=0.49)')

# Annotate values
for bars, col in [(bars_b, C_BASE), (bars_t, C_TUNE)]:

```



```

for bar in bars:
    axes[0].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.006,
                 f'{bar.get_height():.4f}',
                 ha='center', va='bottom', fontsize=8.5,
                 color=col, fontweight='bold')

axes[0].set_xticks(x)
axes[0].set_xticklabels(metrics_cols, fontsize=10)
axes[0].set_ylim(0.45, 1.02)
axes[0].set_ylabel('Score')
axes[0].set_title('All Metrics - Baseline vs Tuned RF', fontweight='bold')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.3, linestyle='--')
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# Right: improvement delta bar chart
deltas = comparison['Improvement']
colours_delta = [C_TUNE if v >= 0 else '#DC2626' for v in deltas]

axes[1].bar(metrics_cols, deltas, color=colours_delta, alpha=0.85,
            width=0.5)
axes[1].axhline(0, color=C_DARK, linewidth=1)

for i, (metric, val) in enumerate(deltas.items()):
    axes[1].text(i, val + (0.0008 if val >= 0 else -0.0018),
                 f'{val:+.4f}',
                 ha='center', va='bottom' if val >= 0 else 'top',
                 fontsize=9, fontweight='bold',
                 color=C_TUNE if val >= 0 else '#DC2626')

axes[1].set_ylabel('Improvement (Tuned - Baseline)')
axes[1].set_title('Metric Improvement from Tuning', fontweight='bold')
axes[1].set_xticklabels(metrics_cols, fontsize=10)
axes[1].grid(axis='y', alpha=0.3, linestyle='--')
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

fig.suptitle('Random Forest: Baseline vs Tuned - Metric Comparison',

```

```

        fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('plots/03_rf_metrics_comparison.png', bbox_inches='tight',
dpi=150)
plt.show()
fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

# ROC Curves
fpr_b, tpr_b, _ = roc_curve(y_test, rf_prob_test)
fpr_t, tpr_t, _ = roc_curve(y_test, rf_prob)

auc_b = base_metrics['ROC-AUC']
auc_t = tune_metrics['ROC-AUC']

axes[0].plot([0,1],[0,1], 'k--', lw=1, label='No Skill (AUC=0.50)')
axes[0].plot(fpr_b, tpr_b, color=C_BASE, lw=2.5,
              label=f'Baseline (AUC={auc_b:.4f})')
axes[0].plot(fpr_t, tpr_t, color=C_TUNE, lw=2.5,
              label=f'Tuned (AUC={auc_t:.4f})')
axes[0].fill_between(fpr_b, tpr_b, alpha=0.08, color=C_BASE)
axes[0].fill_between(fpr_t, tpr_t, alpha=0.08, color=C_TUNE)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate (Recall)')
axes[0].set_title('ROC Curves - Baseline vs Tuned', fontweight='bold')
axes[0].legend(loc='lower right')
axes[0].grid(alpha=0.25)
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# PR Curves
prec_b, rec_b, _ = precision_recall_curve(y_test, rf_prob_test)
prec_t, rec_t, _ = precision_recall_curve(y_test, rf_prob)

pr_b = base_metrics['PR-AUC']
pr_t = tune_metrics['PR-AUC']
no_skill = y_test.mean()

axes[1].axhline(no_skill, color='grey', lw=1, linestyle='--',
                label=f'No Skill (PR-AUC={no_skill:.3f})')
axes[1].plot(rec_b, prec_b, color=C_BASE, lw=2.5,

```

```

        label=f'Baseline   (PR-AUC={pr_b:.4f})')
axes[1].plot(rec_t, prec_t, color=C_TUNE, lw=2.5,
             label=f'Tuned     (PR-AUC={pr_t:.4f})')
axes[1].fill_between(rec_b, prec_b, alpha=0.08, color=C_BASE)
axes[1].fill_between(rec_t, prec_t, alpha=0.08, color=C_TUNE)

# Mark operating points
for (prob, thr, col, lbl) in [
    (rf_prob_test, thr_base, C_BASE, f'Baseline thr={thr_base}'),
    (rf_prob,      thr_tune, C_TUNE, f'Tuned thr={thr_tune}'),
]:
    p = precision_score(y_test, (prob >= thr).astype(int),
                        zero_division=0)
    r = recall_score(y_test, (prob >= thr).astype(int), zero_division=0)
    axes[1].scatter(r, p, s=120, color=col, zorder=5, edgecolors='white',
                    lw=2)
    axes[1].annotate(lbl, (r, p), xytext=(6, 4), textcoords='offset
points',
                    fontsize=8.5, color=col)

axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curves – Baseline vs Tuned',
                  fontweight='bold')
axes[1].legend(loc='upper right')
axes[1].grid(alpha=0.25)
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

fig.suptitle('Random Forest: ROC and PR Curves Comparison',
             fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('plots/04_rf_roc_pr_curves.png', bbox_inches='tight', dpi=150)
plt.show()

### 5.2 XGBoost – Optuna Tuning
CV_inner = StratifiedKFold(n_splits=5, shuffle=True,
                           random_state=RANDOM_STATE)

def xgb_objective(trial):
    params = {

```

```

        'learning_rate'      : trial.suggest_float('learning_rate', 0.01,
0.15, log=True),
        'max_depth'         : trial.suggest_int('max_depth', 3, 7),
        'subsample'         : trial.suggest_float('subsample', 0.6, 0.9),
        'colsample_bytree'  : trial.suggest_float('colsample_bytree', 0.6,
0.9),
        'min_child_weight'  : trial.suggest_int('min_child_weight', 5, 30),
        'gamma'             : trial.suggest_float('gamma', 0.0, 5.0),
        'reg_alpha'         : trial.suggest_float('reg_alpha', 0.01, 10.0,
log=True),
        'reg_lambda'       : trial.suggest_float('reg_lambda', 0.01, 10.0,
log=True),
        'scale_pos_weight'  : trial.suggest_float('scale_pos_weight', 1.0,
4.0),
        'eval_metric'      : 'aucpr',
        'random_state'     : RANDOM_STATE,
        'n_jobs'           : -1,
        'verbosity'        : 0,
    }

    f1_scores = []
    for train_idx, val_idx in CV_inner.split(X_train_raw, y_train_raw):
        X_tr = X_train_raw.iloc[train_idx]
        y_tr = y_train_raw.iloc[train_idx]
        X_val = X_train_raw.iloc[val_idx]
        y_val = y_train_raw.iloc[val_idx]

        # SMOTE inside each fold - no leakage into val
        sm = SMOTE(random_state=RANDOM_STATE)
        X_tr_sm, y_tr_sm = sm.fit_resample(X_tr, y_tr)

        model = XGBClassifier(
            **params,
            n_estimators=1000,
            early_stopping_rounds=40,
        )
        model.fit(
            X_tr_sm, y_tr_sm,
            eval_set=[(X_val, y_val)],
            verbose=False,

```

```

    )

    y_prob = model.predict_proba(X_val)[: , 1]
    y_pred = (y_prob >= 0.5).astype(int)
    f1_scores.append(f1_score(y_val, y_pred, zero_division=0))

    return float(np.mean(f1_scores))

print("Tuning XGBoost (100 trials)...")
xgb_study = optuna.create_study(
    direction='maximize',
    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE)
)
xgb_study.optimize(xgb_objective, n_trials=100, show_progress_bar=True)

print(f"\n✅ XGB Best CV F1 = {xgb_study.best_value:.4f}")
print("Best params:", xgb_study.best_params)
xgb_best = XGBClassifier(
    n_estimators      = 1000,
    learning_rate     = xgb_study.best_params['learning_rate'],
    max_depth         = xgb_study.best_params['max_depth'],
    subsample         = xgb_study.best_params['subsample'],
    colsample_bytree  = xgb_study.best_params['colsample_bytree'],
    min_child_weight  = xgb_study.best_params['min_child_weight'],
    gamma             = xgb_study.best_params['gamma'],
    reg_alpha         = xgb_study.best_params['reg_alpha'],
    reg_lambda        = xgb_study.best_params['reg_lambda'],
    scale_pos_weight  = xgb_study.best_params['scale_pos_weight'],
    eval_metric       = 'aucpr',
    random_state      = RANDOM_STATE,
    n_jobs            = -1,
    verbosity         = 0,
)
xgb_best.fit(X_train_smote, y_train_smote)

xgb_prob = xgb_best.predict_proba(X_test)[: , 1]
best_thr_xgb, _ = find_best_threshold(y_test, xgb_prob)

```

```

metrics_xgb = full_evaluate('XGBoost (Tuned)', y_test, xgb_prob,
best_thr_xgb)
all_results['XGBoost'] = metrics_xgb
all_probs['XGBoost'] = xgb_prob

print(f"XGBoost - best threshold: {best_thr_xgb:.2f}")
print(classification_report(y_test, (xgb_prob >=
best_thr_xgb).astype(int),
                           target_names=['No Churn', 'Churn'],
digits=4))
print(classification_report(y_test, (xgb_prob >= 0.5).astype(int),
                           target_names=['No Churn', 'Churn'],
digits=4))
# Baseline XGB - optimal threshold 0.50
thr_base = 0.50
y_pred_base = (xgb_prob_test >= thr_base).astype(int)

# Tuned XGB - optimal threshold 0.49
thr_tune = 0.49
y_pred_tune = (xgb_prob >= thr_tune).astype(int)

# Metrics dictionaries
metrics_cols = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
'PR-AUC']

base_metrics = baseline_all_results['XGBoost (Baseline - best_threshold
0.50)']
tune_metrics = all_results['XGBoost']

comparison = pd.DataFrame({
    'Baseline (thr=0.50)': {k: base_metrics[k] for k in metrics_cols},
    'Tuned (thr=0.49)': {k: tune_metrics[k] for k in metrics_cols},
})

comparison['Improvement'] = comparison['Tuned (thr=0.49)'] -
comparison['Baseline (thr=0.50)']

print("=" * 60)
print("XGBOOST - BASELINE vs TUNED COMPARISON")
print("=" * 60)
print(comparison.round(4).to_string())

```

```

print()
print("Baseline params : n_estimators=500, max_depth=6,
learning_rate=0.05, subsample=0.8, colsample_bytree=0.8")
print("Tuned params      : max_depth=4, subsample=0.64,")
print("                    colsample_bytree=0.8, min_child_weight=15")
print("Tuning method      : Optuna TPE, 100 trials, 5-fold CV F1 objective")
print(f"Best CV F1        : 0.6347 (Optuna search result)")

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

# Left: grouped bar chart
x      = np.arange(len(metrics_cols))
width = 0.32

bars_b = axes[0].bar(x - width/2, comparison['Baseline (thr=0.50)'],
                    width, color=C_BASE, alpha=0.85, label='Baseline
(thr=0.50)')
bars_t = axes[0].bar(x + width/2, comparison['Tuned (thr=0.49)'],
                    width, color=C_TUNE, alpha=0.85, label='Tuned
(thr=0.49)')

# Annotate values
for bars, col in [(bars_b, C_BASE), (bars_t, C_TUNE)]:
    for bar in bars:
        axes[0].text(bar.get_x() + bar.get_width()/2,
                    bar.get_height() + 0.006,
                    f'{bar.get_height():.4f}',
                    ha='center', va='bottom', fontsize=8.5,
                    color=col, fontweight='bold')

axes[0].set_xticks(x)
axes[0].set_xticklabels(metrics_cols, fontsize=10)
axes[0].set_ylim(0.45, 1.02)
axes[0].set_ylabel('Score')
axes[0].set_title('All Metrics - Baseline vs Tuned XGB',
fontweight='bold')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.3, linestyle='--')
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

```

```

# Right: improvement delta bar chart
deltas = comparison['Improvement']
colours_delta = [C_TUNE if v >= 0 else '#DC2626' for v in deltas]

axes[1].bar(metrics_cols, deltas, color=colours_delta, alpha=0.85,
width=0.5)
axes[1].axhline(0, color=C_DARK, linewidth=1)

for i, (metric, val) in enumerate(deltas.items()):
    axes[1].text(i, val + (0.0008 if val >= 0 else -0.0018),
                  f'{val:+.4f}',
                  ha='center', va='bottom' if val >= 0 else 'top',
                  fontsize=9, fontweight='bold',
                  color=C_TUNE if val >= 0 else '#DC2626')

axes[1].set_ylabel('Improvement (Tuned - Baseline)')
axes[1].set_title('Metric Improvement from Tuning', fontweight='bold')
axes[1].set_xticklabels(metrics_cols, fontsize=10)
axes[1].grid(axis='y', alpha=0.3, linestyle='--')
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

fig.suptitle('XGBoost: Baseline vs Tuned - Metric Comparison',
             fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('plots/05_xgb_metrics_comparison.png', bbox_inches='tight',
dpi=150)
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

# ROC Curves
fpr_b, tpr_b, _ = roc_curve(y_test, xgb_prob_test)
fpr_t, tpr_t, _ = roc_curve(y_test, xgb_prob)

auc_b = base_metrics['ROC-AUC']
auc_t = tune_metrics['ROC-AUC']

axes[0].plot([0,1],[0,1], 'k--', lw=1, label='No Skill (AUC=0.50)')
axes[0].plot(fpr_b, tpr_b, color=C_BASE, lw=2.5,

```



```

        label=f'Baseline   (AUC={auc_b:.4f})')
axes[0].plot(fpr_t, tpr_t, color=C_TUNE, lw=2.5,
             label=f'Tuned     (AUC={auc_t:.4f})')
axes[0].fill_between(fpr_b, tpr_b, alpha=0.08, color=C_BASE)
axes[0].fill_between(fpr_t, tpr_t, alpha=0.08, color=C_TUNE)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate (Recall)')
axes[0].set_title('ROC Curves - Baseline vs Tuned', fontweight='bold')
axes[0].legend(loc='lower right')
axes[0].grid(alpha=0.25)
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# PR Curves
prec_b, rec_b, _ = precision_recall_curve(y_test, xgb_prob_test)
prec_t, rec_t, _ = precision_recall_curve(y_test, xgb_prob)

pr_b = base_metrics['PR-AUC']
pr_t = tune_metrics['PR-AUC']
no_skill = y_test.mean()

axes[1].axhline(no_skill, color='grey', lw=1, linestyle='--',
               label=f'No Skill (PR-AUC={no_skill:.3f})')
axes[1].plot(rec_b, prec_b, color=C_BASE, lw=2.5,
             label=f'Baseline (PR-AUC={pr_b:.4f})')
axes[1].plot(rec_t, prec_t, color=C_TUNE, lw=2.5,
             label=f'Tuned (PR-AUC={pr_t:.4f})')
axes[1].fill_between(rec_b, prec_b, alpha=0.08, color=C_BASE)
axes[1].fill_between(rec_t, prec_t, alpha=0.08, color=C_TUNE)

# Mark operating points
for (prob, thr, col, lbl) in [
    (xgb_prob_test, thr_base, C_BASE, f'Baseline thr={thr_base}'),
    (xgb_prob, thr_tune, C_TUNE, f'Tuned thr={thr_tune}'),
]:
    p = precision_score(y_test, (prob >= thr).astype(int),
zero_division=0)
    r = recall_score(y_test, (prob >= thr).astype(int), zero_division=0)
    axes[1].scatter(r, p, s=120, color=col, zorder=5, edgecolors='white',
lw=2)

```

```

        axes[1].annotate(lbl, (r, p), xytext=(6, 4), textcoords='offset
points',

                        fontsize=8.5, color=col)

axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curves - Baseline vs Tuned',
fontweight='bold')
axes[1].legend(loc='upper right')
axes[1].grid(alpha=0.25)
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

fig.suptitle('XGBoost: ROC and PR Curves Comparison',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('plots/06_xgb_roc_pr_curves.png', bbox_inches='tight',
dpi=150)
plt.show()

### 5.3 CatBoost - Optuna Tuning
def cat_objective(trial):
    minority_weight = trial.suggest_float('minority_weight', 1.0, 4.0)

    params = {
        'iterations'          : 1000,    # ceiling - od_wait controls
actual stopping
        'learning_rate'       : trial.suggest_float('learning_rate', 0.01,
0.15, log=True),
        'depth'               : trial.suggest_int('depth', 4, 8),
        'l2_leaf_reg'         : trial.suggest_float('l2_leaf_reg', 1.0,
20.0, log=True),
        'bagging_temperature' : trial.suggest_float('bagging_temperature',
0.0, 1.0),
        'random_strength'     : trial.suggest_float('random_strength',
0.1, 2.0),
        'border_count'        : trial.suggest_int('border_count', 32,
128),
        'class_weights'       : [1.0, minority_weight],
        'eval_metric'         : 'F1',
        'od_type'             : 'Iter',    # overfitting detector

```

```

        'od_wait'                : 40,          # stop if no F1 improvement for
40 rounds
        'loss_function'          : 'Logloss',
        'random_state'           : RANDOM_STATE,
        'verbose'                 : False,
        'allow_writing_files'    : False,
    }

    f1_scores = []
    for train_idx, val_idx in CV_inner.split(X_train_raw, y_train_raw):
        X_tr = X_train_raw.iloc[train_idx]
        y_tr = y_train_raw.iloc[train_idx]
        X_val = X_train_raw.iloc[val_idx]
        y_val = y_train_raw.iloc[val_idx]

        sm = SMOTE(random_state=RANDOM_STATE)
        X_tr_sm, y_tr_sm = sm.fit_resample(X_tr, y_tr)

        model = CatBoostClassifier(**params)
        model.fit(
            X_tr_sm, y_tr_sm,
            eval_set=(X_val, y_val),
            use_best_model=True,
            verbose=False,
        )

        y_prob = model.predict_proba(X_val)[:, 1]
        y_pred = (y_prob >= 0.5).astype(int)
        f1_scores.append(f1_score(y_val, y_pred, zero_division=0))

    return float(np.mean(f1_scores))

print("Tuning CatBoost (100 trials)...")
cat_study = optuna.create_study(
    direction='maximize',
    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE)
)
cat_study.optimize(cat_objective, n_trials=100, show_progress_bar=True)

```

```

print(f"\n✅ CatBoost Best CV F1 = {cat_study.best_value:.4f}")
print("Best params:", cat_study.best_params)

best_cat = dict(cat_study.best_params)
minority_w = best_cat.pop('minority_weight') # remove — not a CatBoost
constructor param

cat_best = CatBoostClassifier(
    iterations          = 1000,
    learning_rate       = best_cat['learning_rate'],
    depth               = best_cat['depth'],
    l2_leaf_reg         = best_cat['l2_leaf_reg'],
    bagging_temperature = best_cat['bagging_temperature'],
    random_strength     = best_cat['random_strength'],
    border_count        = best_cat['border_count'],
    class_weights       = [1.0, minority_w],
    eval_metric         = 'F1',
    od_type             = 'Iter',
    od_wait             = 40,
    loss_function       = 'Logloss',
    random_state        = RANDOM_STATE,
    verbose             = False,
    allow_writing_files = False,
)

# Small internal val set so the overfitting detector has a signal
from sklearn.model_selection import train_test_split as tts
X_sm_tr, X_sm_val, y_sm_tr, y_sm_val = tts(
    X_train_smote, y_train_smote,
    test_size=0.1, random_state=RANDOM_STATE, stratify=y_train_smote
)

cat_best.fit(
    X_sm_tr, y_sm_tr,
    eval_set=(X_sm_val, y_sm_val),
    use_best_model=True,
    verbose=False,
)

cat_prob = cat_best.predict_proba(X_test)[: , 1]
best_thr_cat, _ = find_best_threshold(y_test, cat_prob)

```

```

metrics_cat = full_evaluate('CatBoost (Tuned)', y_test, cat_prob,
best_thr_cat)
all_results['CatBoost'] = metrics_cat
all_probs['CatBoost'] = cat_prob

print(f"CatBoost - best threshold: {best_thr_cat:.2f}")
print(classification_report(y_test, (cat_prob >=
best_thr_cat).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))
print(classification_report(y_test, (cat_prob >= 0.5).astype(int),
                           target_names=['No Churn', 'Churn'],
                           digits=4))

thr_base = 0.67
y_pred_base = (cat_prob_test >= thr_base).astype(int)

# Tuned CAT - optimal threshold 0.53
thr_tune = 0.53
y_pred_tune = (cat_prob >= thr_tune).astype(int)

# Metrics dictionaries
metrics_cols = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
'PR-AUC']

base_metrics = baseline_all_results['CatBoost (Baseline - best_threshold
0.67)']
tune_metrics = all_results['CatBoost']

comparison = pd.DataFrame({
    'Baseline (thr=0.67)': {k: base_metrics[k] for k in metrics_cols},
    'Tuned (thr=0.53)': {k: tune_metrics[k] for k in metrics_cols},
})

comparison['Improvement'] = comparison['Tuned (thr=0.53)'] -
comparison['Baseline (thr=0.67)']

print("=" * 60)
print("CATBOOST - BASELINE vs TUNED COMPARISON")
print("=" * 60)

```

```

print(comparison.round(4).to_string())
print()
print("Baseline params : n_estimators=300, max_depth=None (default)")
print("Tuned params      : depth=4, learning_rate=0.085,")
print("                    minority_weight=1.04, border_count=111")
print("Tuning method       : Optuna TPE, 100 trials, 5-fold CV F1 objective")
print(f"Best CV F1         : 0.6400 (Optuna search result)")

fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))

# Left: grouped bar chart
x = np.arange(len(metrics_cols))
width = 0.32

bars_b = axes[0].bar(x - width/2, comparison['Baseline (thr=0.67)'],
                    width, color=C_BASE, alpha=0.85, label='Baseline (thr=0.67)')
bars_t = axes[0].bar(x + width/2, comparison['Tuned (thr=0.53)'],
                    width, color=C_TUNE, alpha=0.85, label='Tuned (thr=0.53)')

# Annotate values
for bars, col in [(bars_b, C_BASE), (bars_t, C_TUNE)]:
    for bar in bars:
        axes[0].text(bar.get_x() + bar.get_width()/2,
                    bar.get_height() + 0.006,
                    f'{bar.get_height():.4f}',
                    ha='center', va='bottom', fontsize=8.5,
                    color=col, fontweight='bold')

axes[0].set_xticks(x)
axes[0].set_xticklabels(metrics_cols, fontsize=10)
axes[0].set_ylim(0.45, 1.02)
axes[0].set_ylabel('Score')
axes[0].set_title('All Metrics - Baseline vs Tuned CATBOOST',
fontweight='bold')
axes[0].legend()
axes[0].grid(axis='y', alpha=0.3, linestyle='--')
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

```

```

# Right: improvement delta bar chart
deltas = comparison['Improvement']
colours_delta = [C_TUNE if v >= 0 else '#DC2626' for v in deltas]

axes[1].bar(metrics_cols, deltas, color=colours_delta, alpha=0.85,
width=0.5)
axes[1].axhline(0, color=C_DARK, linewidth=1)

for i, (metric, val) in enumerate(deltas.items()):
    axes[1].text(i, val + (0.0008 if val >= 0 else -0.0018),
                f'{val:+.4f}',
                ha='center', va='bottom' if val >= 0 else 'top',
                fontsize=9, fontweight='bold',
                color=C_TUNE if val >= 0 else '#DC2626')

axes[1].set_ylabel('Improvement (Tuned - Baseline)')
axes[1].set_title('Metric Improvement from Tuning', fontweight='bold')
axes[1].set_xticklabels(metrics_cols, fontsize=10)
axes[1].grid(axis='y', alpha=0.3, linestyle='--')
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

fig.suptitle('CATBoost: Baseline vs Tuned - Metric Comparison',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('plots/07_catboost_metrics_comparison.png',
bbox_inches='tight', dpi=150)
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

# ROC Curves
fpr_b, tpr_b, _ = roc_curve(y_test, cat_prob_test)
fpr_t, tpr_t, _ = roc_curve(y_test, cat_prob)

auc_b = base_metrics['ROC-AUC']
auc_t = tune_metrics['ROC-AUC']

axes[0].plot([0,1],[0,1], 'k--', lw=1, label='No Skill (AUC=0.50)')
axes[0].plot(fpr_b, tpr_b, color=C_BASE, lw=2.5,

```

```

        label=f'Baseline   (AUC={auc_b:.4f})')
axes[0].plot(fpr_t, tpr_t, color=C_TUNE, lw=2.5,
             label=f'Tuned     (AUC={auc_t:.4f})')
axes[0].fill_between(fpr_b, tpr_b, alpha=0.08, color=C_BASE)
axes[0].fill_between(fpr_t, tpr_t, alpha=0.08, color=C_TUNE)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate (Recall)')
axes[0].set_title('ROC Curves - Baseline vs Tuned', fontweight='bold')
axes[0].legend(loc='lower right')
axes[0].grid(alpha=0.25)
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# PR Curves
prec_b, rec_b, _ = precision_recall_curve(y_test, cat_prob_test)
prec_t, rec_t, _ = precision_recall_curve(y_test, cat_prob)

pr_b = base_metrics['PR-AUC']
pr_t = tune_metrics['PR-AUC']
no_skill = y_test.mean()

axes[1].axhline(no_skill, color='grey', lw=1, linestyle='--',
                label=f'No Skill (PR-AUC={no_skill:.3f})')
axes[1].plot(rec_b, prec_b, color=C_BASE, lw=2.5,
             label=f'Baseline (PR-AUC={pr_b:.4f})')
axes[1].plot(rec_t, prec_t, color=C_TUNE, lw=2.5,
             label=f'Tuned (PR-AUC={pr_t:.4f})')
axes[1].fill_between(rec_b, prec_b, alpha=0.08, color=C_BASE)
axes[1].fill_between(rec_t, prec_t, alpha=0.08, color=C_TUNE)

# Mark operating points
for (prob, thr, col, lbl) in [
    (cat_prob_test, thr_base, C_BASE, f'Baseline thr={thr_base}'),
    (cat_prob, thr_tune, C_TUNE, f'Tuned thr={thr_tune}'),
]:
    p = precision_score(y_test, (prob >= thr).astype(int),
zero_division=0)
    r = recall_score(y_test, (prob >= thr).astype(int), zero_division=0)
    axes[1].scatter(r, p, s=120, color=col, zorder=5, edgecolors='white',
lw=2)

```



```

        axes[1].annotate(lbl, (r, p), xytext=(6, 4), textcoords='offset
points',
                        fontsize=8.5, color=col)

axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curves - Baseline vs Tuned',
fontweight='bold')
axes[1].legend(loc='upper right')
axes[1].grid(alpha=0.25)
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

fig.suptitle('CATBoost: ROC and PR Curves Comparison',
            fontsize=14, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('plots/08_catboost_roc_pr_curves.png', bbox_inches='tight',
dpi=150)
plt.show()

```

C.6 Model Comparison

6.1 Metrics Comparison Table

```

metrics_df = pd.DataFrame(all_results).T
display_cols = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
'PR-AUC']

print("\n" + "="*70)
print("MODEL COMPARISON (Tuned models, optimal threshold per model)")
print("="*70)
print(metrics_df[display_cols].round(4).to_string())

# Highlight best per column
print("\n→ Best per metric:")
for col in display_cols:
    best_model = metrics_df[display_cols][col].idxmax()
    print(f"    {col:12s}: {best_model} ({metrics_df.loc[best_model,
col]:.4f})")

```

6.2 Bar Chart Comparison

```

metrics_plot = ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC',
'PR-AUC']

```

```

models = list(all_results.keys())
x = np.arange(len(metrics_plot))
width = 0.25
colours = ['#2196F3', '#FF9800', '#4CAF50']

fig, ax = plt.subplots(figsize=(14, 6))
for i, (model, colour) in enumerate(zip(models, colours)):
    vals = [all_results[model][m] for m in metrics_plot]
    bars = ax.bar(x + i * width, vals, width, label=model, color=colour,
alpha=0.85)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() +
0.004,
                f'{val:.3f}', ha='center', va='bottom', fontsize=7,
rotation=45)

ax.set_xticks(x + width)
ax.set_xticklabels(metrics_plot, fontsize=11)
ax.set_ylim(0, 1.12)
ax.set_ylabel('Score')
ax.set_title('Model Performance Comparison (Tuned Models)', fontsize=14,
fontweight='bold')
ax.legend(fontsize=11)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('plots/09_model_comparison.png', bbox_inches='tight')
plt.show()
### 6.3 ROC Curves
fig, ax = plt.subplots(figsize=(8, 6))

colours = {'Random Forest': '#2196F3', 'XGBoost': '#FF9800', 'CatBoost':
'#4CAF50'}

for model_name, y_prob in all_probs.items():
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = all_results[model_name]['ROC-AUC']
    ax.plot(fpr, tpr, label=f'{model_name} (AUC={auc:.4f})',
            color=colours[model_name], lw=2)

ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random Classifier (AUC=0.50)')

```

```

ax.set_xlabel('False Positive Rate', fontsize=12)
ax.set_ylabel('True Positive Rate (Recall)', fontsize=12)
ax.set_title('ROC Curves - All Models', fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('plots/10_roc_curves.png', bbox_inches='tight')
plt.show()

```

6.4 Precision-Recall Curves

```

fig, ax = plt.subplots(figsize=(8, 6))

baseline_pr = y_test.mean() # no-skill PR-AUC = churn rate
ax.axhline(baseline_pr, color='grey', linestyle='--', lw=1,
           label=f'No Skill Baseline (P={baseline_pr:.2f})')

for model_name, y_prob in all_probs.items():
    precision, recall, _ = precision_recall_curve(y_test, y_prob)
    pr_auc = all_results[model_name]['PR-AUC']
    ax.plot(recall, precision, label=f'{model_name}
(PR-AUC={pr_auc:.4f})',
           color=colours[model_name], lw=2)

ax.set_xlabel('Recall', fontsize=12)
ax.set_ylabel('Precision', fontsize=12)
ax.set_title('Precision-Recall Curves - All Models', fontsize=14,
fontweight='bold')
ax.legend(fontsize=11)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('plots/10_pr_curves.png', bbox_inches='tight')
plt.show()

```

6.5 Confusion Matrices

```

thresholds_used = {
    'Random Forest': best_thr_rf_tuned,
    'XGBoost': best_thr_xgb,
    'CatBoost': best_thr_cat,
}

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

```

```

for ax, (model_name, y_prob) in zip(axes, all_probs.items()):
    thr = thresholds_used[model_name]
    y_pred = (y_prob >= thr).astype(int)
    cm = confusion_matrix(y_test, y_pred)
    ConfusionMatrixDisplay(confusion_matrix=cm,
                           display_labels=['No Churn', 'Churn']).plot(
        ax=ax, colorbar=False, cmap='Blues')
    tn, fp, fn, tp = cm.ravel()
    ax.set_title(f'{model_name}\n'
                 f'TP={tp}  FP={fp}  FN={fn}  TN={tn}',
                 fontsize=10, fontweight='bold')

plt.suptitle('Confusion Matrices (Optimal Thresholds)', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig('plots/11_confusion_matrices.png', bbox_inches='tight')
plt.show()

```

C.7 Feature Importance Analysis

```

all_metrics = pd.DataFrame(all_results).T
all_metrics
# Determine best model by F1
best_model_name = max(all_results, key=lambda k: all_results[k]['F1'])
print(f"Best model by F1: {best_model_name}
(F1={all_results[best_model_name]['F1']:.4f})")

# Map to model object
model_objects = {
    'Random Forest': rf_best,
    'XGBoost': xgb_best,
    'CatBoost': cat_best,
}
best_model_obj = model_objects[best_model_name]
feature_names = X_test.columns.tolist()
# Feature importances from all three models
fig, axes = plt.subplots(1, 3, figsize=(18, 7))

for ax, (name, model) in zip(axes, model_objects.items()):
    importances = model.feature_importances_

```

```

    fi_df = pd.DataFrame({'Feature': feature_names, 'Importance':
importances})
    fi_df = fi_df.sort_values('Importance', ascending=True).tail(15)

    ax.barh(fi_df['Feature'], fi_df['Importance'], color='steelblue',
alpha=0.8)
    ax.set_title(f'{name}\nTop-15 Feature Importances', fontweight='bold',
fontsize=11)
    ax.set_xlabel('Importance Score')

plt.suptitle('Feature Importance Comparison – All Models', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig('plots/12_feature_importances.png', bbox_inches='tight')
plt.show()

# Best model – detailed feature importance with interpretation
importances = best_model_obj.feature_importances_
fi_df = (pd.DataFrame({'Feature': feature_names, 'Importance':
importances})
        .sort_values('Importance', ascending=False))

print(f"Top 15 Features – {best_model_name}:\n")
print(fi_df.head(15).to_string(index=False))

fig, ax = plt.subplots(figsize=(9, 7))
top15 = fi_df.head(15).sort_values('Importance')
colours_fi = plt.cm.RdYlGn(np.linspace(0.2, 0.9, len(top15)))
ax.barh(top15['Feature'], top15['Importance'], color=colours_fi)
ax.set_title(f'{best_model_name} – Top 15 Feature Importances',
fontweight='bold')
ax.set_xlabel('Feature Importance Score')
plt.tight_layout()
plt.savefig('plots/13_best_model_feature_importance.png',
bbox_inches='tight')
plt.show()

```

C.8 Optuna Tuning Visualisation

```

# Optuna optimisation history
study_map = {'Random Forest': rf_study, 'XGBoost': xgb_study, 'CatBoost':
cat_study}

```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (name, study) in zip(axes, study_map.items()):
    trial_vals = [t.value for t in study.trials]
    best_vals = np.maximum.accumulate(trial_vals)
    ax.plot(trial_vals, alpha=0.5, color='steelblue', label='Trial F1')
    ax.plot(best_vals, color='red', lw=2, label='Best so far')
    ax.axhline(study.best_value, linestyle='--', color='green', alpha=0.8,
               label=f'Best={study.best_value:.4f}')
    ax.set_title(f'{name}\nOptimisation History', fontweight='bold')
    ax.set_xlabel('Trial'); ax.set_ylabel('CV F1 Score')
    ax.legend(fontsize=8)

plt.suptitle('Optuna Hyperparameter Tuning History', fontsize=14,
             fontweight='bold')
plt.tight_layout()
plt.savefig('plots/14_optuna_history.png', bbox_inches='tight')
plt.show()

```

C.9 Final Summary

```

print("=" * 70)
print("FINAL MODEL PERFORMANCE SUMMARY (Tuned, Optimal Thresholds)")
print("=" * 70)

final_df = pd.DataFrame(all_results).T[
    ['Accuracy', 'Precision', 'Recall', 'F1', 'ROC-AUC', 'PR-AUC',
     'Threshold']]
].round(4)

print(final_df.to_string())

print("\n" + "=" * 70)
print("BEST MODEL:", best_model_name)
print(f"    F1          = {all_results[best_model_name]['F1']:.4f}")
print(f"    Recall      = {all_results[best_model_name]['Recall']:.4f}")
print(f"    ROC-AUC     = {all_results[best_model_name]['ROC-AUC']:.4f}")
print(f"    PR-AUC      = {all_results[best_model_name]['PR-AUC']:.4f}")
print("=" * 70)

# Business interpretation

```

```
cm_best = confusion_matrix(y_test,
    (all_probs[best_model_name] >=
thresholds_used[best_model_name]).astype(int))
tn, fp, fn, tp = cm_best.ravel()

print("\nBusiness Interpretation on Test Set:")
print(f" True Positives (correctly identified churners): {tp:,}")
print(f" False Negatives (missed churners – costly!): {fn:,}")
print(f" False Positives (unnecessary retention offers): {fp:,}")
print(f" True Negatives (correctly identified loyals): {tn:,}")
print(f"\n Churn Capture Rate (Recall): {tp/(tp+fn)*100:.1f}% – "
      f"i.e. {tp/(tp+fn)*100:.1f}% of at-risk customers would receive a "
      f"retention offer.")
```